

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. АНАЛИТИЧЕСКИЙ ОБЗОР	5
1.2. Описание предметной области.....	5
1.3. Основание для разработки.....	6
1.4. Сравнительный анализ.....	7
1.4.1. Jenkins	7
1.4.2. GitLab CI/CD	8
1.4.3. Spinnaker	8
1.4.4. GitHub Actions.....	9
1.5. Назначение разработки	10
1.6. Постановка задачи	11
2. ПРОЕКТИРОВАНИЕ ИС	13
2.1. Формирования требований к системе	13
2.2. Выбор и описание средств программной реализации	14
2.2.1. Terraform.....	14
2.2.2. Jenkins	15
2.2.3. Docker.....	15
2.2.4. k3s.....	16
2.2.5. Prometheus и Grafana	17
2.2.6. Python и Pygame	17
2.3. Модель компонентов системы и их взаимодействие.....	18
2.3.1. Конфигурация развёртывания и автоматизации	18
2.3.2. Формирование конфигураций инфраструктуры и мониторинга...	19
2.3.3. Система симуляции и логика регулирования	20
2.4. Моделирование процессов развёртывания и работы.....	21
2.4.1. Последовательность выполнения конвейеров.....	21
2.5. Диаграмма последовательности.....	23
2.6. Диаграмма вариантов использования.....	24
2.7. Требования к безопасности и отказоустойчивости	24
3. РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ.....	26
3.1. Настройка мастер-сервера	26
3.2. Автоматизированное развёртывание инфраструктуры	26
3.2.1. Подготовка конфигурации Terraform	26

3.2.2. Создание облачных виртуальных машин	28
3.2.3. Хранение состояния и интеграция с Jenkins.....	30
3.2.4. Формирование выходных данных и передача в другие этапы	31
3.3. Построение конвейеров CI/CD в Jenkins.....	33
3.3.1. Конвейер проверки инфраструктуры	33
3.3.2. Конвейер перехода в продуктивную среду.....	34
3.4. Автоматизация мониторинга.....	36
3.4.1. Установка node_exporter и сбор данных	36
3.4.2. Генерация Ansible inventory через Bash	38
3.4.3. Подключение Prometheus.....	40
3.4.4. Настройка отображения графиков.....	41
3.5. Установка и настройка инструментов автоматизации	43
3.6. Разработка симуляции транспортного движения.....	45
3.7. Сборка Docker-образа и публикация в Docker Hub	47
3.8. Развёртывание подов симуляции на удалённых k3s-нодах	49
ЗАКЛЮЧЕНИЕ	53

ВВЕДЕНИЕ

В наши дни все больше разрабатываются и эксплуатируются информационные системы (ИС), имеющих совершенно различные области автоматизации, от систем поддержки принятия решений до различного рода управленческих или экспертных ИС. Если некоторые из них канут в лету, то оставшиеся разрастутся до невероятных размеров, и получают не только клиентов и долю на рынке, но и сопутствующие проблемы, такие как: сложная составляющая развертывания и масштабирования. Поэтому при разработке следует задумываться не только над программным кодом и выходным результатом, но и над возможностью дальнейшей модернизации ИС, что подразумевает под собой упрощенное выстраивание новых компонентов, а также увеличение их “плотности” в компьютерной среде [14][15].

Существует огромное количество платформ, помогающие внедрять ИС. Они предоставляют колоссальные возможности, с помощью которых можно реализовывать различного рода решения, подходящие по требованиям и потребностям любого предприятия, а также имеют всю необходимую гибкость, масштабируемость, интегрируемость, удобство и безопасность.

Если не самой важной, то точно одной из самых востребованных является сфера бизнеса. Существует множество типовых подходов к созданию информационных систем, предназначенных для учета, планирования и контроля в компаниях различных отраслей и масштабов. Однако, несмотря на широкий спектр методологий, некоторые из них не всегда будут являться подходящими, а иногда и вовсе не учесть специфику работы конкретного предприятия. В подобных ситуациях возникает потребность в более новейших решениях, которые бы учитывали не только особенности организационной структуры, специфику бизнес-процессов, методы анализа данных, а также потребность к постоянному росту, быстрому внедрению новых функций и быстрому восстановлению после сбоя.

Поэтому особенно важным аспектом является автоматизация развертывания информационной системы, с чем помогает справиться методология DevOps(Development Operations). Данный метод использует решения CI/CD (Continuous Integration/Continuous delivery), то есть непрерывную интеграцию и непрерывную доставку функций системы. Компании, внедрившие соответствующую философию в свои практики, являются более успешными на фоне тех, кто этого не сделал [13].

1. АНАЛИТИЧЕСКИЙ ОБЗОР

1.1. Описание предметной области

Предметной областью работы является разработка решения, направленного на автоматизированное развертывания, масштабирование, мониторинг, и непрерывную доставку функций системы предприятия с использованием специализированных инструментов, и построение системы с использованием декларативного подхода.

За основу взята система имитационного моделирования транспортного движения, реализующая перекрёсток с регулируемыми направлениями и автоматической системой управления светофорами. В данной модели учитываются различные аспекты уличного движения, такие как плотность потока, время отклика светофоров, адаптация фаз переключения и логика приоритетов в зависимости от интенсивности с каждой стороны. Визуализация осуществляется средствами графического интерфейса, что позволяет наблюдать за динамикой транспортных потоков в режиме реального времени.

Информационно-симуляционная система предназначена для отладки и анализа поведения транспортной сети в условиях переменной загруженности. Она позволяет проводить эксперименты по изменению параметров сигналов, выявлять узкие места, тестировать различные сценарии пропускной способности, а также настраивать алгоритмы адаптивного управления без риска для реального движения. Модель даёт возможность централизованно отслеживать параметры движения автомобилей, сигналы светофоров и статистику по количеству проезжающих машин. Это делает её полезным инструментом как для образовательных целей, так и для инженерной апробации решений в области транспортного моделирования.

Для успешного внедрения соответствующей методологии на базе такой структуры документооборота, система не просто должна обладать рядом

ключевых характеристик, таких как удобство интерфейса, высокая производительность, надежность хранения данных, гибкость настройки и возможность масштабирования под изменяющиеся потребности. Также очень важно придерживаться всех ключевых аспектов философии DevOps, где основными процессами будут:

1. Доставка — непрерывная настройка, развертывание инфраструктуры и функций приложения и отправка их в рабочую среду;
2. Операции — обслуживание системы с целью увеличения надежности, предотвращения неполадок;
3. Мониторинг — наблюдение за конвейерами DevOps, а также за служебными метриками системы.

1.2. Основание для разработки

Основанием для разработки является развитие практики DevOps в России при быстро меняющиеся реалиях к методологии сопровождения ИС.

За последние годы в сфере информационных систем произошли значительные трансформации, связанные с повышением требований к гибкости и адаптивности IT-решений. Несмотря на прогресс в разработке программного обеспечения, многие организации продолжают сталкиваться с трудностями при сопровождении ИС, вынужденно используя ручные методы настройки, обновления и масштабирования инфраструктуры. Это приводит к увеличению времени на внедрение изменений, росту ошибок из-за человеческого фактора и ограничению возможностей быстрого реагирования на меняющиеся бизнес-потребности.

В связи с потребностью в оптимизации управления IT-инфраструктурой возникла необходимость разработки решений, позволяющих систематизировать процессы сопровождения ИС. Автоматизация развертывания, благодаря контейнеризации и оркестрации,

обеспечивает единообразие сред разработки и эксплуатации. Это не только упрощает масштабирование ресурсов под текущие задачи, но и снижает риски возникновения конфликтов в конфигурациях, что особенно важно для предприятий, работающих в распределенных экосистемах.

В России соответствующая философия не так сильно развита, если сравнивать ее со странами западной Европы или США. Основными причинами является непонимание компаниями эффективности ее внедрения из-за низкой публицистической активности, недостаточности достоверной информации и огромным выбором технологий, сложностью их реализации и интеграции, что является серьезной проблемой для предприятий в плане обучения персонала.

1.3. Сравнительный анализ

Рассмотрим основные существующие средства реализации информационных систем, которые предоставляют функции для автоматизации настройки развертывания, масштабирования и мониторинга.

1.3.1. Jenkins

является мощным инструментом непрерывной доставки и интеграции, предназначенный для сквозной автоматизации. Одним из значительных достоинств этой системы является высокая гибкость настроек, позволяющая предприятиям полностью адаптировать систему под индивидуальные потребности, включая разработку конвейеров CI/CD при помощи языка DSL(Domain-Specific Language), что помогает улучшить процесс сборки при помощи кода. Кроме того, программа имеет совместимость с различными операционными системами, такими как: Windows, Linux и macOS. Позволяет визуализировать процессы сборки, используя графическое представление конвейеров и различных таблиц с информацией. Программа является

полностью бесплатной, обладает открытым исходным кодом и огромным количеством дополнительных плагинов.

Тем не менее, система имеет и недостатки. Самая большая сложность этой системы — необходимость в серьёзных знаниях и практическом опыте при настройке. Также данный инструмент достаточно ресурсоемкий при большом количестве задач, что требует дополнительной оптимизации.

1.3.2. GitLab CI/CD

По набору возможностей GitLab CI/CD во многом схож с Jenkins, но работает немного иначе. Она позволяет автоматизировать сборку и развертывание кода. Инструмент не требует дополнительных серверов и работает через облако. Также GitLab использует синтаксис YAML (YAML Ain't Markup Language), который проще по сравнению с DSL Jenkins. Может быть интегрирован с облачными платформами, поддерживает масштабируемый и гибкий. Встроенные возможности мониторинга и безопасности, имеет современный интерфейс, что включает отчеты о покрытии тестами и анализ времени и доставки изменений.

Главным минусом данной системы является ограниченность ее бесплатной версии. GitLab имеет меньшую гибкость по сравнению с Jenkins, а также зависимость от собственного сервера.

1.3.3. Spinnaker

Spinnaker — это платформа, рассчитанная, в основном, на непрерывную доставку. Специализируется на развертывании в облаках и кластерах продуктов оркестрации, в частности Kubernetes. Может поддерживать мульти-облачные развертывания, а также гибкие стратегии доставки, такие как: быстрое переключение между двумя версиями, пошаговое обновление без времени простоя. Система совместима со многими популярными средствами непрерывной интеграции, такими как: Jenkins,

GitLab, GitHub Actions, а также с инструментами управления инфраструктурой: Terraform, Helm.

Spinnaker имеет ряд ограничений. Система требует крайне высоких навыков, особенно для настройки небольших проектов. Нужны высокие навыки облачных архитектур. Также потребляет много ресурсов CPU и памяти, требует несколько собственных серверов, как для системных процессов инструмента, так и для баз собственных данных. Крайне слабо развито сообщество, вследствие чего наблюдается меньше готовых решений и не сильно развитую документацией в отличии от других систем CI/CD.

1.3.4. GitHub Actions

GitHub Actions представляет собой мощную встроенную платформу для автоматизации процессов непрерывной интеграции и доставки (CI/CD), предлагая гибкие возможности для оптимизации рабочих процессов разработки. Его универсальность позволяет адаптировать систему под самые разнообразные проекты.

Одним из ключевых преимуществ GitHub Actions является его глубокая интеграция с экосистемой GitHub, что делает процесс настройки CI/CD максимально удобным. Рабочие процессы можно описывать в виде конфигураций на YAML, что упрощает создание и чтение настроек, использовать готовые шаблоны или создавать собственные Actions для автоматизации рутинных задач. Это заметно сокращало время на настройку и облегчало переход от разработки к запуску.

Однако, несмотря на свою гибкость, GitHub Actions может потребовать некоторого времени на освоение, особенно для команд, которые ранее не работали с подобными системами. Хотя платформа предлагает интуитивно понятный интерфейс и обширную документацию, новые пользователи могут столкнуться с необходимостью изучения

особенностей конфигурационных файлов, управления артефактами и настройки собственных раннеров.

В результате сравнительного анализа была выбрана система Jenkins, это обосновано рядом ключевых факторов. Инструмент отличается высокой степенью гибкости, позволяет адаптировать его под уникальные требования облачных технологий, а также систем конфигурации инфраструктур, программы контейнеризации и оркестрации, включая возможность добавления огромного количества дополнительных модулей. Организации получают доступ к мощному инструменту автоматизации без дополнительных затрат на подписки или платные модули. Все основные функции, включая сборку и развёртывание, доступны "из коробки", а расширение возможностей достигается за счёт бесплатных плагинов из обширного сообщества. Большое сообщество DevOps-инженеров, использующих Jenkins, являются значительным активом, облегчающим поиск специалистов для поддержки и улучшения системы, а также разработку необходимого функционала и решению возникающих вопросов.

1.4. Назначение разработки

Объектом разработки является программный комплекс Jenkins, задействованный в построении CI/CD-конвейера, взаимодействующего с моделью транспортного узла.

Функциональным назначением программного изделия является обеспечение возможности автоматизации настройки, развертывания, масштабирования и мониторинга связанных с деятельностью соответствующего предприятия.

Эксплуатационным назначением программного изделия является повышение эффективности, качества и оперативности процессов разработки

новых функций, их развертывания, а также возможность масштабирования и мониторинга ИС.

Особенностью программного изделия является то, что оно имеет открытый исходный код, а также огромное количество дополнительных программных пакетов, которые в полной мере соответствуют философии методологии DevOps.

1.5. Постановка задачи

Для достижения цели работы – разработать структуру сквозной автоматизации для информационной системы управления транспортными потоками необходимо решить следующие задачи:

- Определить функциональные и нефункциональные требования к самому ядру автоматизации информационной системе имитации городского транспортного потока со светофорным управлением;
- Спроектировать метод сквозной автоматизации для информационной системы имитации транспортного узла, основываясь на лучших практиках DevOps, используя инструмент непрерывной интеграции и доставки Jenkins. Выбрать подходящие решения и инструменты для разработки, такие как: Jenkins, Git, GitHub, Docker, Terraform, Prometheus, Grafana и т.д.; [16]
- Оптимизировать конфигурацию системы под нужды предприятия и разработать скрипты непрерывного развертывания, масштабирования, доставки и мониторинга продукта, основываясь на лучших практиках;
- Провести тестирование и отладку симуляционной модели с адаптивной логикой управления светофорами, проверить соответствие требованиям, функциональности и качеству, а также устранить возможные ошибки и сбои в работе;

— Проанализировать риски и ограничения, связанные с созданием и внедрением симуляционной модели транспортного движения. В их числе: необходимость адаптации под различные аппаратные платформы, высокая нагрузка на систему при визуализации в реальном времени, трудности масштабирования при расширении модели, а также возможные проблемы интеграции с облачными средствами мониторинга и автоматического развертывания.

2. ПРОЕКТИРОВАНИЕ ИС

2.1. Формирования требований к системе

Разрабатываемая система предназначена для автоматизированного управления имитацией транспортного потока с применением облачных технологий и сквозной автоматизации.

Функции, которые должна выполнять система:

- Автоматическое создание виртуальных машин в облаке;
- Настройку сетевой инфраструктуры и подключение внешних IP-адресов;
- Установку системных компонентов;
- Развёртывание контейнеризированной симуляции транспортного потока;
- Динамическое управление светофорами в зависимости от загрузки;
- Генерацию визуального интерфейса для мониторинга в реальном времени;
- Централизованный сбор и отображение метрик из всех узлов системы;
- Отказоустойчивость системы: выход из строя одного узла не должен прерывать работу всей симуляции;
- Независимость компонентов: каждая нода должна функционировать автономно;
- Возможность горизонтального масштабирования через изменение конфигурации Terraform;
- Конфигурация и управление системой должны быть реализованы в виде сквозного CI/CD-конвейера;
- Минимизация ручного труда администратора: все шаги выполняются автоматически через Jenkins;

- Доступность визуализации симуляции через noVNC в браузере;
- Сохранение целостности состояния системы между перезапусками и конфигурациями.

2.2. Выбор и описание средств программной реализации

2.2.1. Terraform

Развёртывание виртуальной инфраструктуры с помощью Terraform стало основой всей архитектуры проекта. Этот инструмент выбран не случайно: он предоставляет декларативную модель управления ресурсами, в которой описывается не порядок действий, а желаемое состояние системы. Это упрощает поддержку и позволяет изменять масштаб или структуру инфраструктуры без глубокого погружения в внутренние механизмы облачного провайдера [1][11].

В контексте проекта Terraform используется для создания всех базовых компонентов в Yandex.Cloud : виртуальных машин, сетевых интерфейсов, внешних IP-адресов, подсетей и S3-совместимого хранилища для backend [1] [12].

Благодаря использованию переменных и шаблонов удалось сделать конфигурации универсальными, масштабируемыми и пригодными для повторного использования.

Особое внимание в реализации уделено удалённому хранению состояния. Это позволяет одновременно использовать одни и те же конфигурации из разных конвейеров Jenkins, обеспечивая согласованность и защиту от коллизий. Кроме того, все значения, необходимые для дальнейшей настройки системы, экспортируются и автоматически подставляются в скрипты Ansible и Prometheus, исключая риск ручных ошибок и экономя время.

2.2.2. Jenkins

Вся автоматизация строится вокруг Jenkins, который объединяет в цепочку все этапы — от развёртывания до запуска приложения. Он координирует всю цепочку действий от развёртывания до визуализации. Благодаря этому инструменту удалось создать надёжный и повторяемый процесс, в котором все стадии интеграции, установки и запуска сведены в единую последовательность.

В рамках работы Jenkins подключён к репозиторию, где хранятся все конфигурационные файлы: Terraform, Ansible, Dockerfile и Kubernetes-манифесты. При любом изменении в коде или ручном запуске Jenkins инициирует выполнение одного из конвейеров. Каждый конвейер реализован в виде Groovy-скрипта и состоит из последовательных этапов, которые можно изолированно тестировать и дорабатывать [14][15].

Одним из преимуществ Jenkins стало то, что вся логика CI/CD-циклов реализуется в виде кода — pipeline as code. Это обеспечивает прозрачность, возможность версионирования и, при необходимости, откат к предыдущему состоянию. Jenkins работает на выделенном мастер-сервере и самостоятельно выполняет весь процесс автоматизации, исключая необходимость ручного участия. Процесс масштабируется и легко адаптируется под новые среды или требования [2].

2.2.3. Docker

Для контейнеризации симуляции был выбран Docker [11][12] как наиболее зрелая и устойчивая технология. Он позволил собрать всё приложение — симуляцию движения с визуализацией, адаптивное управление, вспомогательные службы — в единый контейнер, позволяя запускать приложение на любой машине с Docker, обеспечивая быстрое и предсказуемое развёртывание [4].

Особое внимание уделялось сборке образа: он включает не только Python-приложение, но и сервер noVNC, позволяющий подключаться к визуализации через браузер. Внутри контейнера запускается Pygame-программа, а пользователь может взаимодействовать с ней удалённо. Такой подход полностью устраняет необходимость локального GUI или прямого доступа к серверу, что особенно ценно в облачных условиях.

После сборки образ публикуется в Docker Hub. Это обеспечивает централизованное обновление и избавляет от необходимости пересборки на каждой ноде. Jenkins-конвейер автоматически подтягивает актуальную версию контейнера и разворачивает её на рабочем узле, включая в оркестрацию через k3s. Это гарантирует, что все узлы работают с одной версией кода и поведения, а обновление можно проводить без простоя.

2.2.4. k3s

Оркестрация контейнеров в рамках проекта реализована на базе k3s — компактной и производительной реализации Kubernetes, созданной специально для ограниченных и распределённых сред. Это решение оказалось особенно подходящим для нашей архитектуры, где каждое вычислительное ядро — это отдельная виртуальная машина с внешним IP, развернутая в облаке без внутренней сети [7].

Главное достоинство k3s — в том, что он прост в установке и подходит даже для небольших кластеров. Добавление нового узла к кластеру происходит за счёт передачи команды регистрации в процессе установки. Всё, что требуется — это IP-адрес мастера, токен подключения и несколько параметров запуска. Этот процесс был полностью автоматизирован через Ansible и встроен в Jenkins-конвейер, что позволяет автоматически подключать все созданные Terraform-инстансы к кластеру без ручной настройки [7].

После подключения каждая нода становится частью кластера и может принимать workload — в нашем случае, это pod с контейнером симуляции.

Благодаря этому удаётся создать отказоустойчивую схему: если одна нода выходит из строя, другие продолжают работу, а на её место можно быстро развернуть новую.

2.2.5. Prometheus и Grafana

Контроль и визуализация состояния распределённой системы реализованы с помощью Prometheus и Grafana. Эти инструменты были выбраны за счёт своей гибкости, лёгкости интеграции и открытой архитектуры. На каждой виртуальной машине устанавливается `node_exporter` — небольшое приложение, собирающее метрики об использовании CPU, памяти, сетевых интерфейсов и дисковой подсистемы. Prometheus обращается к указанным узлам через определённые интервалы и фиксирует полученные метрики, формируя хронологические записи для последующего анализа [5], [6].

В реализации был применён удобный приём: список IP-адресов всех активных машин формируется автоматически на основе данных от Terraform и сразу же сохраняется в конфигурационный файл Prometheus. Благодаря этому отпадает нужда в ручной настройке — при добавлении новых узлов они автоматически подхватываются системой без вмешательства.

Все собранные показатели выводятся в интерфейсе Grafana, где можно построить графики, задать пороги и просматривать изменения во времени, задать пороги тревоги и визуализировать тенденции. Такой подход облегчает анализ: как только одна из нод начнёт перегружаться, это сразу станет заметно на графике, и можно оперативно принять меры. Кроме того, есть возможность добавить собственные метрики — например, количество машин на перекрёстке — и отслеживать их динамику.

2.2.6. Python и Pygame

Вся логика симуляции была написана на языке Python, а за отображение происходящего отвечает библиотека Pygame. Этот выбор

обусловлен несколькими факторами: доступностью библиотек, активным сообществом и достаточной производительностью для задач визуального моделирования. В центре симуляции находится адаптивная система управления перекрёстком, где светофоры изменяют фазы в зависимости от плотности трафика [8][9].

Алгоритмы учитывают текущее количество машин в зонах детекторов и принимают решения о приоритетах. Благодаря этому симуляция становится ближе к реальным условиям движения в городе. Визуальная часть рисует дороги, автомобили, сигналы светофоров, зоны обнаружения, таймеры и подсказки. Работа Pygame организована в одном основном цикле, который также учитывает состояние системы и отображает его.

Для удалённого доступа используется сервер поVNC, установленный внутри контейнера. Это позволяет любому пользователю с браузером подключиться к симуляции и наблюдать за её работой в реальном времени. Такой подход исключает необходимость установки каких-либо дополнительных приложений на клиентской стороне.

2.3. Модель компонентов системы и их взаимодействие

2.3.1. Конфигурация развёртывания и автоматизации

Организация процесса развёртывания информационной системы опирается на принцип полной автоматизации. Все этапы, начиная с создания виртуальных машин и заканчивая запуском симуляции, выполняются последовательно, без участия администратора. Это стало возможным благодаря тесному взаимодействию между средствами управления инфраструктурой, установкой программного обеспечения и запуском приложения.

Первым шагом служит формирование виртуальной среды в облаке. Все инструкции по созданию инфраструктуры заранее описаны в шаблонах, и

процесс их запуска происходит без участия пользователя. Как только все машины созданы, система передаёт информацию о них, включая IP-адреса, для дальнейшего использования в настройке. Эти данные затем используются для генерации конфигурационных файлов, необходимых для дальнейшей настройки.

После подготовки инфраструктуры запускается процесс установки системного окружения. Каждая машина получает минимально необходимый набор программ: средства наблюдения, платформу контейнеров и компоненты для подключения к управляющему узлу. Подключение осуществляется по внешнему адресу при помощи заранее заданного ключа.

Дополнительно предусмотрена автоматическая настройка системы наблюдения. Список доступных машин формируется на основании полученных ранее данных и сохраняется в конфигурационном файле. Таким образом, при изменении состава узлов все необходимые параметры обновляются без вмешательства пользователя.

Следует отметить, что приведённая схема охватывает только структуру и порядок выполнения. Конкретные сценарии настройки и фрагменты конфигураций приведены в следующей главе, где рассматриваются особенности реализации на практике.

2.3.2. Формирование конфигураций инфраструктуры и мониторинга

После развёртывания виртуальных машин и установки необходимых компонентов система требует согласованной настройки для обеспечения мониторинга и взаимодействия между всеми узлами. Весь процесс происходит автоматически — он использует данные, которые были сформированы ранее при развёртывании окружения.

Для управления настройками создаётся специальный файл, содержащий перечень всех задействованных машин. Он формируется на основе выходных данных системы управления инфраструктурой, где уже отражены адреса и параметры новых узлов. С использованием этой

информации автоматически создаётся инвентарь, необходимый для работы сценариев настройки, а также файл конфигурации наблюдения.

В каждой машине запускается процесс сбора технических данных. Он не требует постоянного контроля, а обновляется при необходимости. Список машин, подлежащих мониторингу, передаётся в систему наблюдения в виде отдельного файла, структура которого соответствует стандарту. Это позволяет обновлять конфигурацию динамически — при добавлении новых машин они сразу становятся доступными для централизованного контроля.

Сама система наблюдения получает сведения о каждой машине: загруженность, доступность, использование ресурсов. Эти данные обрабатываются и визуализируются на центральном сервере в виде наглядных графиков.

2.3.3. Система симуляции и логика регулирования

Ключевым компонентом проектируемой системы является программная модель перекрёстка с регулируемым движением. Она реализована как отдельный модуль, который разворачивается на каждой рабочей машине и функционирует независимо от других узлов. Такое решение позволяет распределить нагрузку и обеспечить устойчивость при выходе из строя отдельных компонентов.

Принцип работы симуляции основан на наблюдении за текущей обстановкой на дорогах и принятии решений о переключении сигналов светофора. В основе модели лежит логика адаптивного регулирования: система измеряет плотность транспортного потока в определённых зонах и на основе полученных данных изменяет продолжительность фаз. Это позволяет сократить задержки, избежать скоплений и приближает поведение симуляции к реальной городской обстановке.

Каждый экземпляр симуляции содержит собственный набор светофоров, автомобилей и детекторов. Светофоры действуют по единому алгоритму, но с учётом локальных условий. Алгоритм анализирует

количество машин в зоне видимости, текущее состояние сигнала и оставшееся время. При достижении пороговых значений происходит переключение фаз — например, продление зелёного сигнала при повышенной нагрузке.

Обмен данными между компонентами симуляции не требуется: каждая нода обрабатывает свой участок трафика. Это решение проще в сопровождении и позволяет легко добавлять новые узлы без необходимости перераспределять работу вручную. В случае увеличения числа узлов нагрузка распределяется естественным образом, поскольку каждая симуляция работает автономно.

Графическая часть реализована внутри контейнера и открывается пользователю через браузер. Отображение включает не только движение транспорта, но и текущее состояние светофоров, счётчики времени, зоны детекторов и другие элементы. Это облегчает отладку и даёт наглядное представление о работе всей системы.

2.4. Моделирование процессов развёртывания и работы

2.4.1. Последовательность выполнения конвейеров

В основе развёртывания и сопровождения системы лежит строго определённая последовательность автоматизированных действий. Эта цепочка запускается по команде пользователя либо автоматически при изменении конфигураций в репозитории. Каждый шаг зависит от результатов предыдущего, что позволяет обеспечить надёжность и предсказуемость при любом запуске.

Процесс начинается с подготовки инфраструктуры. На этом этапе система создаёт все необходимые ресурсы в облаке: виртуальные машины, адреса, сетевые подключения и хранилище. После завершения инициализации формируются выходные данные, которые содержат информацию обо всех созданных узлах. Эти сведения сохраняются и

используются на следующих этапах без необходимости дополнительного ввода.

Далее начинается настройка программного окружения. Каждой машине присваивается роль, после чего запускаются сценарии установки. Они выполняют подключение к центральному серверу, устанавливают средства сбора данных и обеспечивают запуск вспомогательных служб. Особое внимание уделяется конфигурации взаимодействия между узлами: благодаря использованию внешних адресов и шаблонов все параметры подставляются автоматически.

После завершения установки всех компонентов на рабочие машины загружается и разворачивается контейнер с приложением. Его запуск выполняется вместе с системой управления, которая проверяет состояние каждой ноды и направляет задание на соответствующий узел. Контейнер включает в себя как алгоритм симуляции, так и визуальный интерфейс, доступный пользователю.

Наконец, завершающим шагом становится настройка мониторинга и визуализации. Список всех активных машин передаётся в систему наблюдения, где формируются графики и панели. Они позволяют отслеживать состояние инфраструктуры и работу приложения в реальном времени.

2.5. Диаграмма последовательности

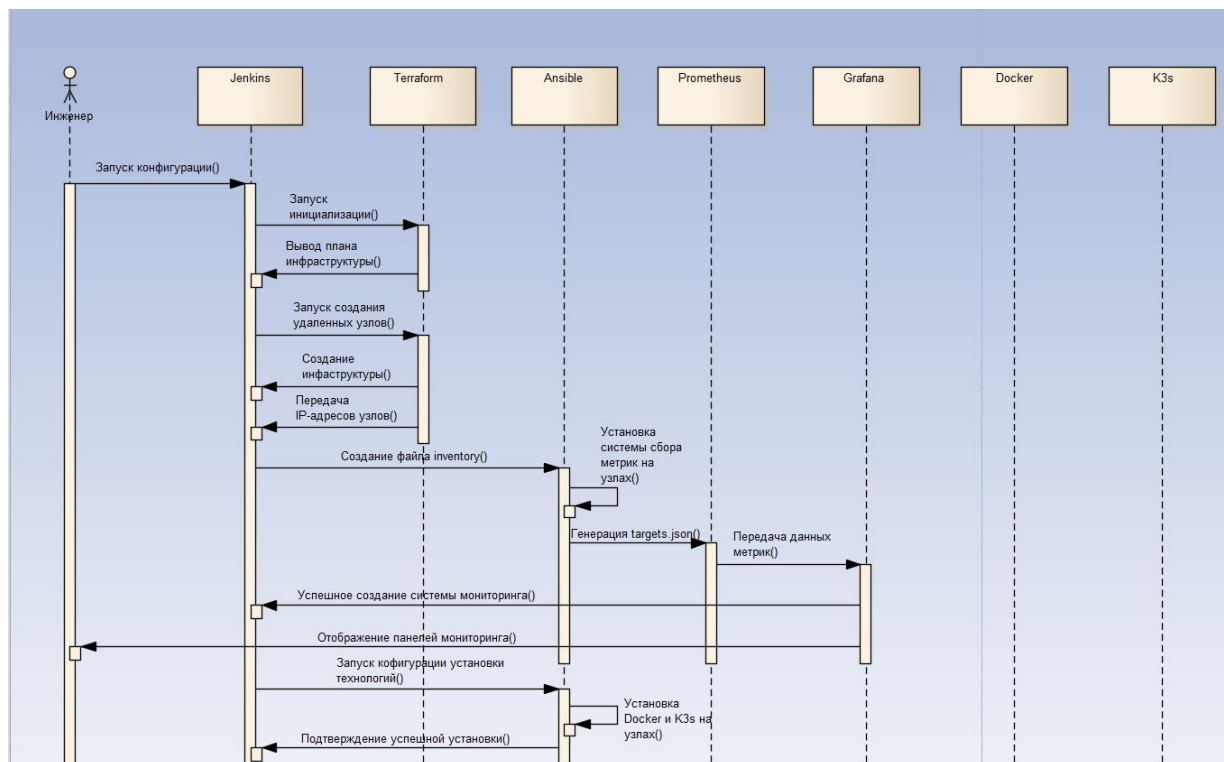


Рис. 1. Диаграмма последовательности

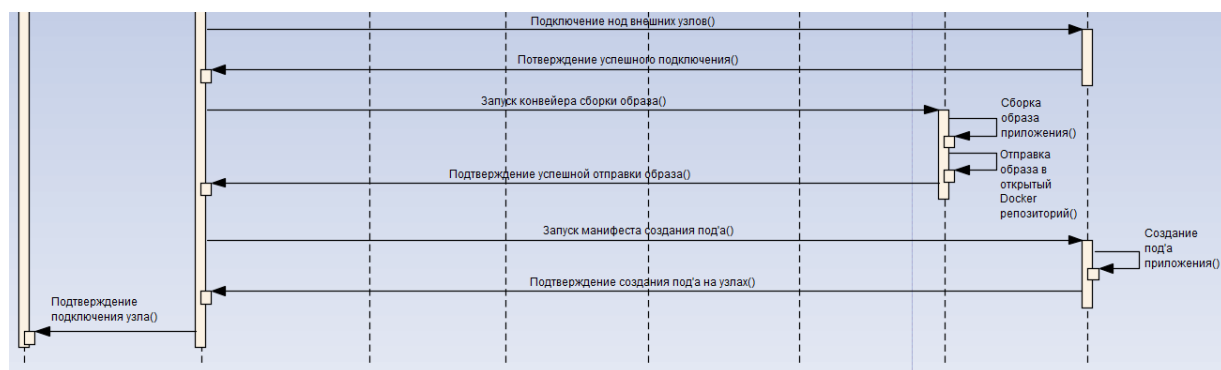


Рис. 1.1. Диаграмма последовательности

2.6. Диаграмма вариантов использования

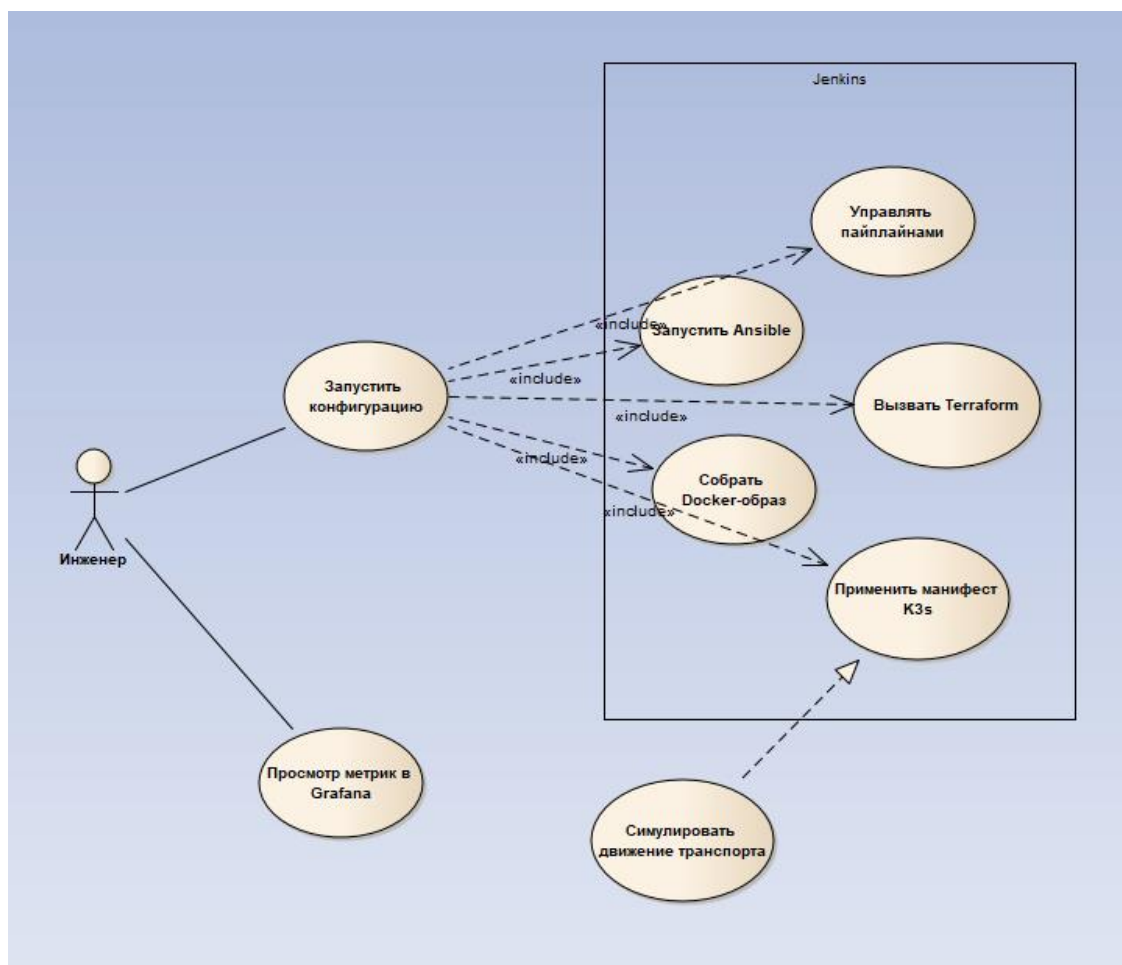


Рис. 2. Диаграмма вариантов использования

2.7. Требования к безопасности и отказоустойчивости

Надёжность и устойчивость работы системы были заложены на этапе проектирования. Виртуальные машины размещаются в разных зонах доступности, что позволяет избежать полной остановки при сбое в одном из сегментов облака. Каждая нода работает независимо: выход из строя одной не влияет на остальные, а новая машина может быть быстро подключена взамен.

Обмен данными между узлами не предусмотрен, что упрощает архитектуру и исключает распространение сбоев. Симуляции запускаются автономно, а мониторинг состояния осуществляется с управляющего сервера, что обеспечивает централизованный контроль.

Доступ к машинам защищён: используется только ключевая аутентификация и зашифрованные соединения. Все порты, не задействованные для работы системы, закрыты. Управляющие интерфейсы недоступны извне и ограничены только необходимыми компонентами.

Информация о конфигурации и состоянии не публикуется открыто. Для хранения используется облачное хранилище с ограниченным доступом. Все параметры передаются по защищённым каналам и используются исключительно в рамках автоматизированных процессов.

За счёт таких решений система остаётся работоспособной при частичных сбоях и защищённой от внешних угроз.

3. РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1. Настройка мастер-сервера

В основе всей системы автоматизации лежит мастер-сервер, который был развёрнут вручную в облаке Yandex.Cloud. В качестве операционной системы выбрана Ubuntu LTS — стабильная и совместимая с требуемыми DevOps-инструментами. На этом сервере последовательно устанавливались Terraform, Docker, Python и Jenkins, а также были созданы отдельные рабочие каталоги под конфигурационные файлы, в частности `/var/lib/jenkins/shared-terraform`.

Особое внимание на начальном этапе было уделено безопасности и предсказуемости работы. Доступ по паролю был отключён, подключение возможно только по SSH-ключу. Все сетевые соединения ограничены базовым файрволом. Jenkins был установлен как системный сервис и настроен на автоматический запуск при старте системы.

Также была подготовлена структура каталогов и настроено окружение для запуска `bash`-скриптов, Terraform-команд и конвейеров. В дальнейшем мастер-сервер использовался для запуска всех этапов автоматизации — от генерации инвентарей до деплоя контейнеров на удалённые узлы.

3.2. Автоматизированное развёртывание инфраструктуры

3.2.1. Подготовка конфигурации Terraform

Для организации виртуальной инфраструктуры в Yandex.Cloud использовался инструмент Terraform, который позволяет описывать все необходимые ресурсы через текстовые файлы и автоматически развёртывать их без ручного вмешательства. Это особенно удобно, когда нужно быстро создавать машины в нескольких зонах и управлять их параметрами централизованно.

Конфигурация была разделена на логические части: в одном файле (`main.tf`) описывались сами виртуальные машины, в другом (`variables.tf`) — все ключевые параметры, которые можно удобно переопределять без изменения основной логики. Для подключения к Yandex.Cloud отдельно указывался провайдер с параметрами доступа — идентификатором облака, папки и файлом сервисного аккаунта.

Одной из ключевых особенностей настройки стала реализация машин через `for_each`. Это нужно чтобы задать список узлов в виде карты и автоматически разворачивать нужное количество экземпляров, каждому присваивая нужную зону и имя. Первые инстансы получали внешний IP, а дублирующие оставались во внутренней сети. Это удобно с точки зрения масштабирования, достаточно добавить ещё одну строку в карту — и Jenkins во время конвейера автоматически создаст дополнительную ноду в облаке.

Кроме машин, в конфигурацию был включён объект `output`, который формировал файл с IP-адресами. Этот файл позже использовался для настройки мониторинга и генерации Ansible-инвентаря. Все данные сохранялись в папке `/var/lib/jenkins/shared-terraform`, чтобы к ним можно было обращаться из разных этапов автоматизации.

Конфигурация разрабатывалась сразу с расчётом на то, что ею будет управлять Jenkins — поэтому пути были прописаны с учётом окружения Jenkins, а входные переменные были совместимы с переменными окружения.

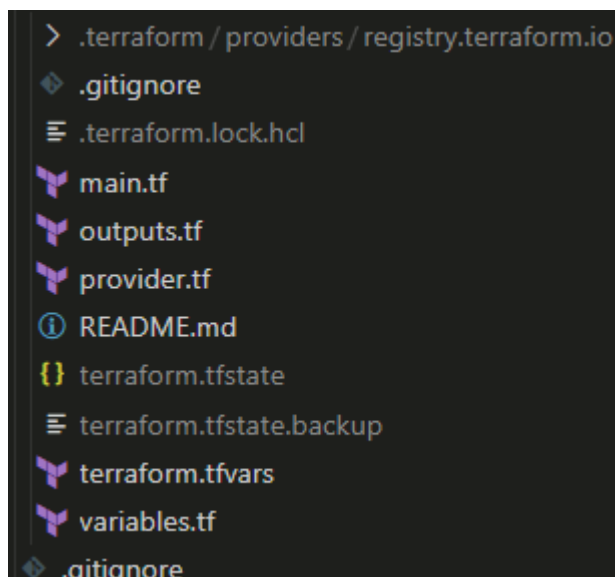


Рис. 3. Директория с файлами Terraform

3.2.2. Создание облачных виртуальных машин

После подготовки всех конфигурационных файлов следующим шагом стало само-развёртывание виртуальных машин в облаке. Для этого использовался набор команд Terraform, запускавшихся как вручную, так и через конвейеры Jenkins в дальнейшем. Основная задача заключалась в том, чтобы создать стабильное распределённое окружение, пригодное для масштабирования и работы в условиях возможных отказов.

При первом запуске конфигурации выполнялась инициализация, в ходе которой Terraform загружал все необходимые провайдеры и проверял структуру модулей. Затем происходил анализ состояния инфраструктуры и сравнение его с описанным в конфигурации. В случае отсутствия ранее развёрнутых ресурсов Terraform сообщал о необходимости создания новых экземпляров.

```

19
20 # Создание дисков и виртуальных машин
21 resource "yandex_compute_disk" "boot_disk" {
22   for_each = local.vm_map
23
24   name      = "${local.boot_disk_name}-${substr(each.value.zone, -1, 0)}-${each.value.index}"
25   zone      = each.value.zone
26   image_id  = var.image_id
27
28   type      = var.instance_resources.disk.disk_type
29   size      = var.instance_resources.disk.disk_size
30 }
31
32 resource "yandex_compute_instance" "this" {
33   for_each = local.vm_map
34
35   name      = "${local.linux_vm_name}-${substr(each.value.zone, -1, 0)}-${each.value.index}"
36
37   allow_stopping_for_update = true
38   platform_id               = var.instance_resources.platform_id
39   zone                      = each.value.zone
40
41   resources {
42     cores    = var.instance_resources.cores
43     memory   = var.instance_resources.memory
44   }
45
46   boot_disk {
47     disk_id = yandex_compute_disk.boot_disk[each.key].id
48   }
49
50   You, 2 months ago | 1 author (You)

```

Рис. 4. Содержание файла main.tf

Все виртуальные машины создавались в рамках заранее определённой карты, где каждая зона получала по одному или нескольким инстансам. В рамках одной зоны только один узел получал внешний IP-адрес, остальные работали через внутреннюю сеть. Это решение обеспечивало минимально необходимый выход наружу и одновременно позволяло организовать доступ ко всем нодам через один публичный адрес при помощи туннелирования или проксирования.

В процессе развёртывания Terraform автоматически присваивал каждому ресурсу уникальные имена, используя индекс в пределах зоны, чтобы избежать конфликтов и сделать результат максимально читаемым в интерфейсе Yandex.Cloud. Также в каждый узел внедрялась публичная часть SSH-ключа, что позволяло сразу после создания виртуальной машины подключаться к ней по ключу — без ручной настройки.

По завершении развёртывания формировался файл с выходными данными, содержащими IP-адреса всех созданных машин. Этот файл позже

использовался как основа для генерации динамического инвентаря и настройки мониторинга. Все данные автоматически сохранялись в общем каталоге shared-terraform, откуда к ним имели доступ другие скрипты и инструменты.

```
iezekil > VKR > erp-terraform > stage > variables.tf > ...
You, 2 months ago | 1 author (You)
1  variable "secret_key" {
2      type      = string
3      sensitive = true
4  }
5
You, 2 months ago | 1 author (You)
6  variable "access_key" {
7      type      = string
8      sensitive = true
9  }
10
You, 2 months ago | 1 author (You)
11 variable "vm_count_per_zone" {
12     description = "Number of VMs per availability zone"
13     type        = number
14     default     = 1
15 }
16
You, 2 months ago | 1 author (You)
17 variable "ssh_public_key" {
18     description = "SSH public key for instance access"
19     type        = string
20 }
21
You, 2 months ago | 1 author (You)
22 variable "name_prefix" {
23     description = "(Optional) - Name prefix for project."
24     type        = string
25     default     = "project"
26 }
27
You, 2 months ago * Add to dev
You, 2 months ago | 1 author (You)
28 variable "boot_disk_name" {
29     description = "(Optional) - Name of the boot disk."
30     type        = string
31     default     = null

```

Рис. 5. Содержание файла variable.tf

Создание происходило быстро и стабильно, а при повторных запусках Terraform корректно обрабатывал уже существующие ресурсы, не создавая дубликатов. Таким образом, была достигнута главная цель этого этапа — развернуть рабочую, гибкую и предсказуемую инфраструктуру, которая не требует ручного вмешательства ни при запуске, ни при масштабировании.

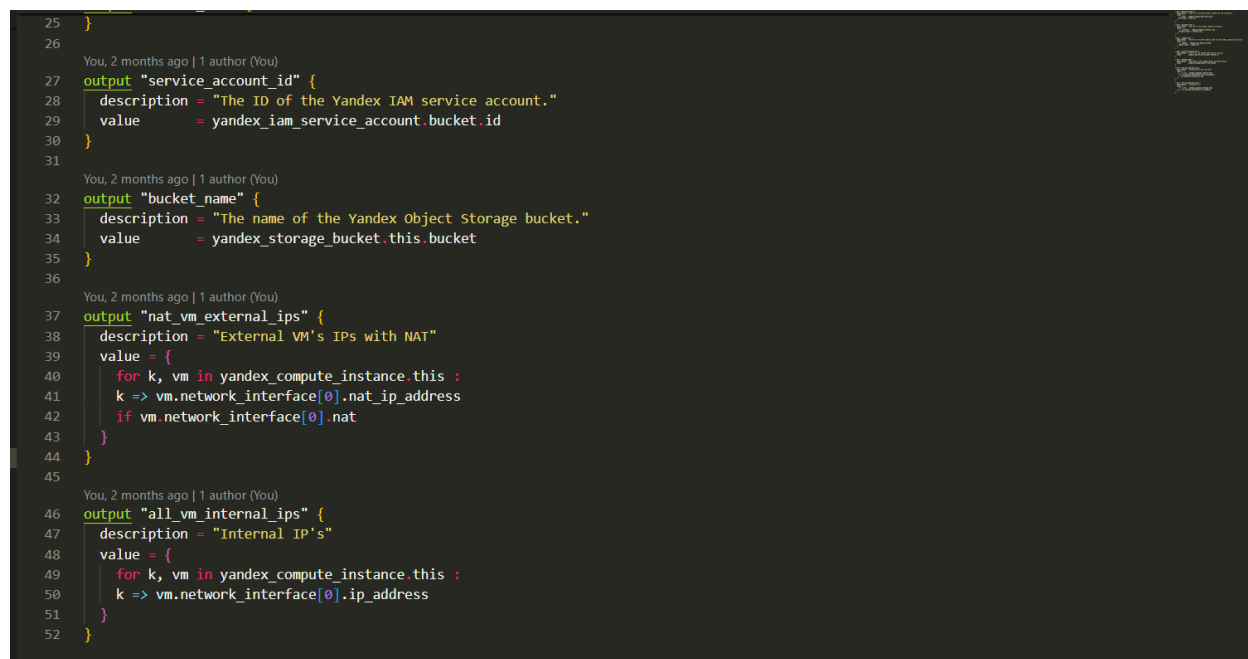
3.2.3. Хранение состояния и интеграция с Jenkins

Для стабильной работы Terraform и возможности повторного запуска конфигураций с сохранением уже созданных ресурсов необходимо было организовать систему хранения состояния. Настройка backend'a

производилась в файле `provider.tf`, где указывались параметры бакета, ключа и региона.

Дополнительно локальная копия состояния и выходных переменных сохранялась в папке `/var/lib/jenkins/shared-terraform`.

Jenkins разворачивает через отдельную стадию конвейера. Запуск Terraform осуществлялся строго в общей рабочей директории, что исключало конфликты и проблемы с путями. Такой подход позволил конвейерам `to-stage` и `to-prod` работать независимо, но с единым backend'ом и синхронизированным состоянием. При этом каждый запуск был детерминированным: если ресурсы уже были созданы, Terraform не выполнял лишних действий. Хранение состояния и его интеграция с Jenkins обеспечили надёжность и повторяемость всех операций, связанных с управлением инфраструктурой.



```
25 }
26
27 You, 2 months ago | 1 author (You)
28 output "service_account_id" {
29   description = "The ID of the Yandex IAM service account."
30   value       = yandex_iam_service_account.bucket_id
31 }
32
33 You, 2 months ago | 1 author (You)
34 output "bucket_name" {
35   description = "The name of the Yandex Object Storage bucket."
36   value       = yandex_storage_bucket.this.bucket
37 }
38
39 You, 2 months ago | 1 author (You)
40 output "nat_vm_external_ips" {
41   description = "External VM's IPs with NAT"
42   value = {
43     for k, vm in yandex_compute_instance.this :
44     k => vm.network_interface[0].nat_ip_address
45     if vm.network_interface[0].nat
46   }
47 }
48
49 You, 2 months ago | 1 author (You)
50 output "all_vm_internal_ips" {
51   description = "Internal IP's"
52   value = {
53     for k, vm in yandex_compute_instance.this :
54     k => vm.network_interface[0].ip_address
55   }
56 }
```

Рис. 6. Содержание файла `outputs.tf`

3.2.4. Формирование выходных данных и передача в другие этапы

После успешного развёртывания инфраструктуры одним из ключевых шагов стало извлечение и использование выходных данных. Terraform по завершении работы автоматически формировал структуру с параметрами

созданных ресурсов — в первую очередь это были IP-адреса всех виртуальных машин. Эти данные нужны были не только для визуального контроля, но и для дальнейших этапов автоматизации.

Чтобы передать эти сведения в другие инструменты, использовалась специальная секция `output` внутри Terraform-конфигурации. В ней задавался список внешних и внутренних IP-адресов, а также другие метки, по которым позже происходила идентификация узлов. Все выходные данные сохранялись в JSON-файл `terraform-outputs.json`, который размещался в директории `/var/lib/jenkins/shared-terraform`. Этот файл стал универсальным источником информации для последующих шагов.

На его основе формировался инвентарный файл для Ansible, который использовался при установке `node_exporter`, Docker и k3s-агентов. Также именно этот JSON-файл обрабатывался `bash`-скриптом при генерации конфигурации Prometheus в формате `file_sd`. Выходные данные Terraform оказались связующим звеном между декларативным описанием инфраструктуры и фактическим сервисов [5][7].

Передача данных происходила автоматически. Скрипты, запускаемые в следующих конвейерах Jenkins, читали `terraform-outputs.json` и преобразовывали его в нужный формат, без ручного вмешательства. Это значительно упростило связку между этапами и исключило типичные ошибки, связанные с копированием IP-адресов вручную или разрозненными конфигурациями.

Выходные переменные стали не просто побочным результатом работы Terraform, а важным каналом связи между различными компонентами системы: инфраструктурой, конфигурационным управлением и мониторингом. Благодаря этому подходу удалось обеспечить цельную, непрерывную цепочку автоматизации от создания виртуальных машин до настройки узлов и развертывания приложений.


```
Stage Logs (Terraform Apply)
Shell Script -- cd ${SHARED_DIR} export YC_TOKEN=${YC_TOKEN} terraform apply -auto-approve -no-color tfplan (self time 1min 51s)
}
}
nat_vm_external_ips = {
  "ru-central1-a-0" = "51.250.69.254"
  "ru-central1-b-0" = "158.160.75.124"
  "ru-central1-d-0" = "158.160.189.52"
}
```

Рис. 7. Вывод файла outputs.tf в Jenkins лог-файле

3.3. Построение конвейеров CI/CD в Jenkins

3.3.1. Конвейер проверки инфраструктуры

Автоматизация развёртывания инфраструктуры начиналась с разработки отдельного конвейера Jenkins, ориентированного на промежуточную среду. Этот конвейер, получивший имя Brasl-ToStage-Terraform, выполнял весь цикл работы с Terraform — от инициализации до применения конфигурации. Основная его задача — гарантировать, что любая новая версия инфраструктуры будет развёрнута корректно, повторяемо и без участия вручную.

Структура конвейера была организована поэтапно. Сначала происходила подготовка окружения: активировался профиль сервисного аккаунта Yandex.Cloud, настраивались переменные окружения, загружались ключи доступа и токен. Это было необходимо для авторизации Terraform при обращении к API облака [1][12].

Далее запускалась стадия инициализации Terraform. На этом шаге происходила проверка структуры конфигурации и загрузка необходимых провайдеров. После этого выполнялся terraform plan — предварительный анализ изменений, отображающий, какие ресурсы будут созданы или изменены. На финальной стадии запускался terraform apply с автоматическим подтверждением.

Весь процесс происходил строго внутри общей директории /var/lib/jenkins/shared-terraform, что позволяло повторно использовать файлы состояния, переменные и выходные данные без их перемещения между разными местами. Благодаря этому удалось исключить нестабильное поведение между запусками и добиться максимальной предсказуемости результата.

Для повышения надёжности были реализованы проверки, контролирующие наличие нужных переменных и токена, а также отслеживание ошибок на каждом этапе. В случае сбоя Jenkins автоматически помечал стадию как ошибочную, а лог выполнения сохранялся для последующего анализа.

Важно, что запуск конвейера происходил вручную из веб-интерфейса Jenkins, но при этом все параметры уже были подготовлены заранее — достаточно было нажать одну кнопку, и весь процесс шёл без вмешательства. Такой подход отлично зарекомендовал себя при разработке, позволяя безопасно тестировать изменения, не затрагивая продуктивную среду.

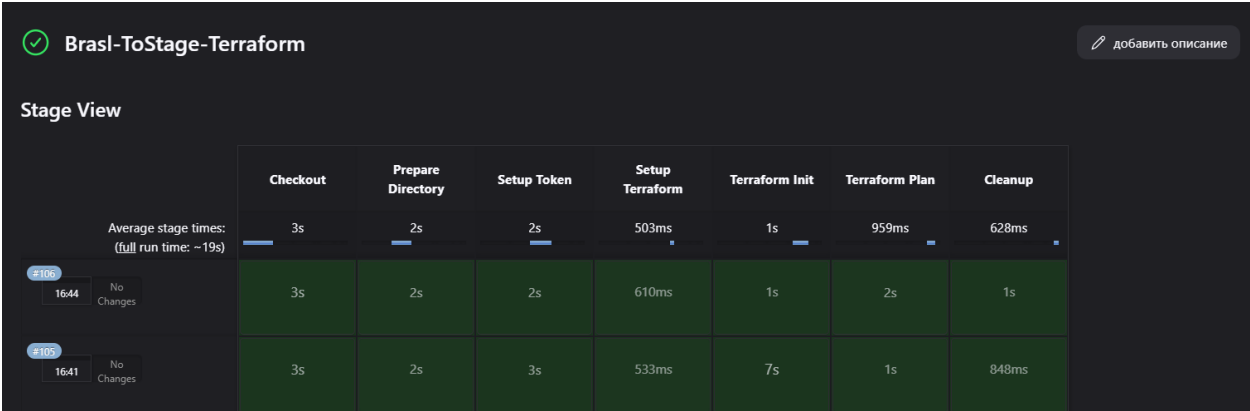


Рис. 8. Конвейер Jenkins для инициализации инфраструктуры

3.3.2. Конвейер перехода в продуктивную среду

После успешного запуска в промежуточной среде инфраструктура должна была быть перенесена в продуктивный контур. Для этого был создан отдельный Jenkins-конвейер Brasl-ToProd-Terraform, работающий по тому же

принципу, что и to-stage, но с особым вниманием к стабильности и повторяемости развёртывания [2].

Главной особенностью этого конвейера стало использование общего backend'а и состояния с промежуточной средой. Благодаря использованию Yandex Object Storage в качестве backend-хранилища, оба конвейера работали с единым файлом terraform.tfstate, но в разных режимах.

Процесс внутри to-prod начинался с подгрузки переменных окружения и токена доступа, после чего выполнялась команда terraform apply. Всё происходило в общем каталоге /var/lib/jenkins/shared-terraform, где уже находились выходные данные и конфигурации, проверенные на предыдущем этапе.

Инфраструктура, протестированная и проверенная в to-stage, могла быть развёрнута идентично в to-prod — без дублирования кода и повторной настройки окружения [16].

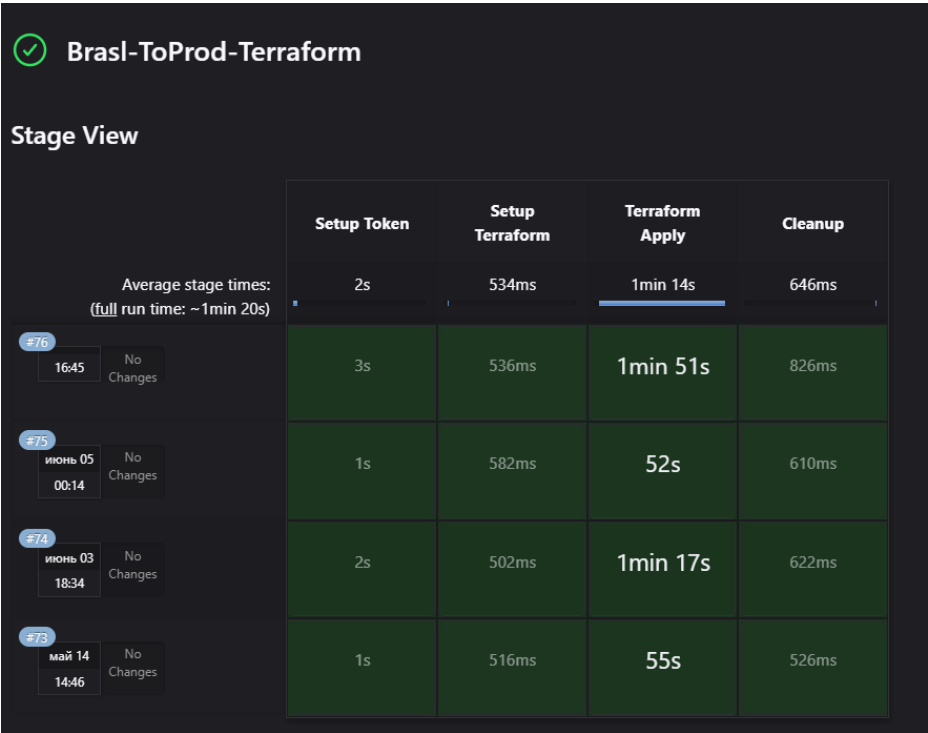


Рис. 9. Конвейер Jenkins для создания инфраструктуры

3.4. Автоматизация мониторинга

3.4.1. Установка `node_exporter` и сбор данных

Для базового мониторинга состояния виртуальных машин использовался `node_exporter` — лёгкий агент от Prometheus, предназначенный для сбора информации о загрузке CPU, оперативной памяти, файловых системах и сетевой активности. Его установка происходила автоматически на каждом узле сразу после их развёртывания, без участия пользователя.

Процесс установки был оформлен в виде отдельного Ansible-плейбука, который запускался из Jenkins при помощи `bash`-скрипта. Скрипт обращался к заранее сгенерированному инвентарному файлу, содержащему список всех машин с внешними IP-адресами. Эти IP-адреса, в свою очередь, формировались из выходных данных Terraform, сохранённых в `terraform-outputs.json`. Установка `node_exporter` стала частью связанного конвейера — от создания инфраструктуры до настройки агентов.

Каждый узел, попавший в инвентарь, получал команду на установку `node_exporter` через стандартный пакетный менеджер. После установки сервис автоматически запускался и начинал прослушивать порт 9100. В конфигурации не требовалась никакая ручная авторизация или ввод паролей — все команды выполнялись через SSH.

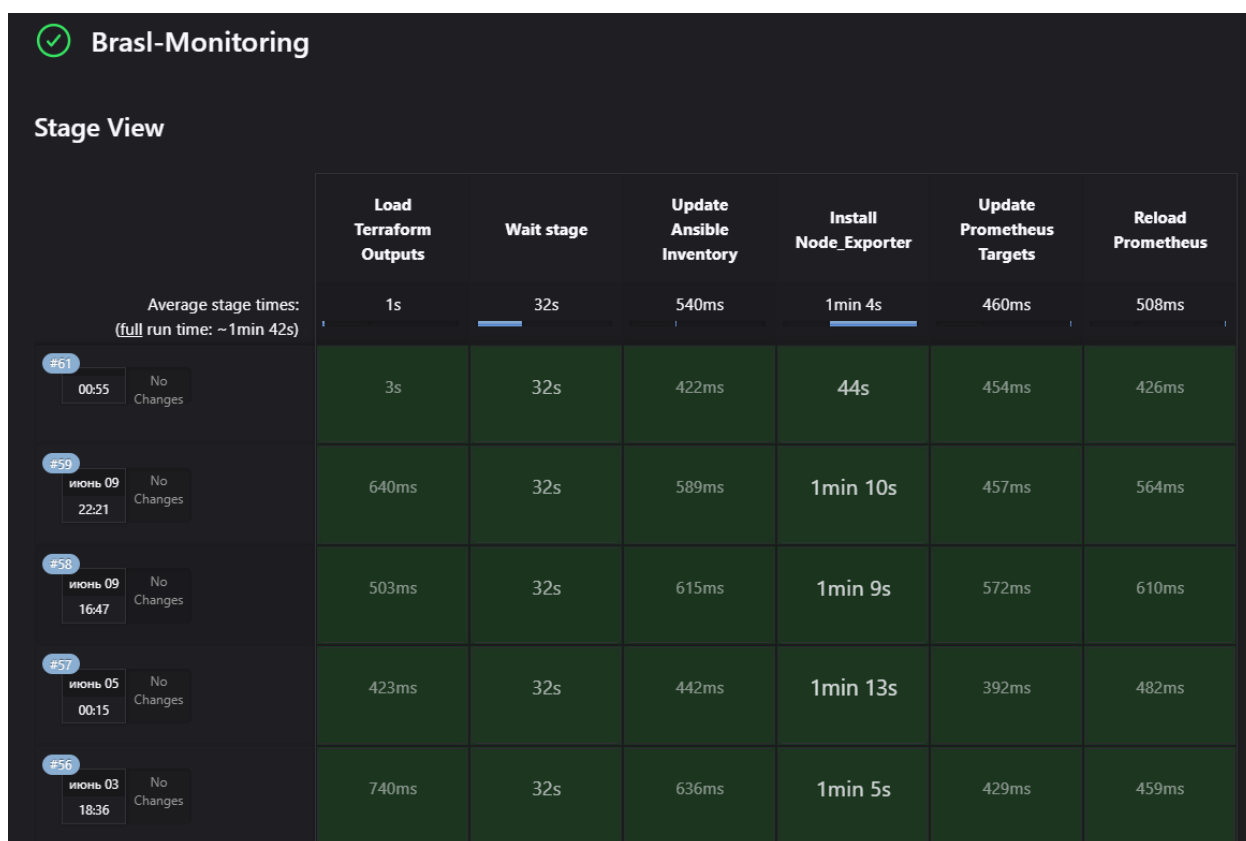


Рис. 10. Конвейер Jenkins для настройки мониторинга узлов

После запуска `node_exporter` стал сразу доступен для Prometheus, который в дальнейшем подключался к каждому из агентов и собирал от них метрики. Каждый инстанс облачной инфраструктуры автоматически попадал под наблюдение, а все данные начинали поступать в систему мониторинга уже в течение первых минут после развёртывания.

Интеграция с Jenkins, Ansible и Terraform позволила сделать установку `node_exporter` полностью автономной. Она происходила каждый раз, когда запускался соответствующий конвейер, и при необходимости могла быть повторена в любой момент без дополнительной настройки.

```
jenkins@brasl-vkr-master:~/ansible$ cat install_node_exporter.yml
---
- name: Install Node_Exporter
  hosts: nodes
  become: true
  tasks:
    - name: Install dependencies
      apt:
        name:
          - wget
          - tar
        state: present
        update_cache: true

    - name: Download Node Exporter
      get_url:
        url: https://github.com/prometheus/node_exporter/releases/download/v1.8.1/node_exporter-1.8.1.linux-amd64.tar.gz
        dest: /tmp/node_exporter.tar.gz

    - name: Extract Node Exporter
      unarchive:
        src: /tmp/node_exporter.tar.gz
        dest: /opt/
        remote_src: yes

    - name: Create systemd unit file for node_exporter
      copy:
        dest: /etc/systemd/system/node_exporter.service
        content: |
          [Unit]
          Description=Node Exporter
          After=network.target

          [Service]
          User=root
          ExecStart=/opt/node_exporter-1.8.1.linux-amd64/node_exporter
          Restart=always

          [Install]
          WantedBy=default.target
      notify:
        - Restart node_exporter

  handlers:
    - name: Restart node_exporter
      systemd:
        name: node_exporter
        enabled: true
        state: restarted
        daemon_reload: true
```

Рис. 11. Содержание Ansible playbook по установке node_exporter

3.4.2. Генерация Ansible inventory через Bash

Для того чтобы Ansible мог взаимодействовать с виртуальными машинами, ему необходимо передавать адреса узлов в специальном формате — так называемом инвентаре. Поскольку адреса создаются динамически на этапе развёртывания, было важно автоматизировать формирование этого файла, чтобы он всегда содержал актуальные данные. Для этой задачи был написан небольшой bash-скрипт `generate_inventory.sh`, запускавшийся сразу после применения Terraform.

Скрипт обращался к файлу выходных данных (terraform-outputs.json), в котором хранились IP-адреса машин, созданных на предыдущем этапе. С помощью утилиты jq он извлекал только те IP, которые относились к инстансам с внешним адресом (NAT). На их основе формировался простой текстовый файл в формате Ansible, где каждая строка содержала IP и имя пользователя для подключения — в данном случае ubuntu.

Файл инвентаря сохранялся в директории /var/lib/jenkins/ansible/inventory, откуда его сразу подхватывали плейбуки. Такой подход позволил запускать любые задачи Ansible (например, установку node_exporter, Docker или k3s) без ручной корректировки адресов, даже если инфраструктура только что была пересоздана или масштабирована.

Автоматическая генерация инвентаря устранила проблему человеческого фактора: не нужно было копировать IP вручную, забывать про обновления или случайно использовать старый список. Каждый раз скрипт заново формировал список узлов, и система работала только с актуальными данными.

Эта связка между Terraform, JSON-выходами и Ansible является важной частью всей архитектуры. Она позволяет добиться гибкости, когда инфраструктура может меняться — добавляться, удаляться, обновляться — а инструменты настройки всегда будут знать, к каким хостам нужно подключаться и что на них выполнять.

```
iezekii > VKR > bash_scripts > $ generate_inventory.sh
You, 2 months ago | 1 author (You)
1  #!/bin/bash
2  set -e
3
4  SHARED_DIR="/var/lib/jenkins/shared-terraform"
5  ANSIBLE_DIR="/var/lib/jenkins/ansible"
6
7  echo "Generating Ansible inventory from Terraform outputs..."
8
9  cd "$SHARED_DIR"
10
11 # Create inventory file with nodes group
12 {
13   echo "[nodes]"
14   jq -r '.nat_vm_external_ips.value | to_entries[] | "\(.value) ansible_user=ubuntu"' terraform-outputs.json
15 } > "${ANSIBLE_DIR}/inventory"
16
17 echo "Ansible inventory successfully created at ${ANSIBLE_DIR}/inventory"
18
```

Рис. 12. Содержание Bash скрипта по созданию Ansible inventory

3.4.3. Подключение Prometheus

После установки `node_exporter` на все виртуальные машины необходимо было подключить систему мониторинга. В качестве основного средства сбора метрик использовался Prometheus — мощный инструмент, позволяющий регулярно опрашивать удалённые источники и сохранять собранные данные для анализа. Основной задачей на этом этапе стало автоматическое добавление новых машин в конфигурацию Prometheus без необходимости вручную прописывать адреса.

Для этого применялась возможность Prometheus под названием `file-based service discovery (file_sd)`. Она позволяет указывать список целей мониторинга во внешнем JSON-файле, который Prometheus регулярно перечитывает.

Был написан отдельный скрипт `generate_instances.sh`, который, как и генератор инвентаря, извлекал IP-адреса, но уже в формате, пригодном для Prometheus. Каждый IP оборачивался в структуру JSON с указанием порта 9100 и меткой `node_exporter`, после чего результат сохранялся в `/opt/prometheus/file_sd/instances.json`.

Prometheus был настроен на использование именно этого файла. Конфигурация включала блок с путём к `instances.json`, и по умолчанию система проверяла его каждые 30 секунд. Как только появлялся новый IP, он сразу попадал в список целей для опроса.


```

iezekiil > VKR > bash_scripts > $ generate_instances.sh
You, 2 months ago | 1 author (You)
1  #!/bin/bash
2  set -e
3
4  SHARED_DIR="/var/lib/jenkins/shared-terraform"
5  OUTPUT_FILE="/opt/prometheus/file_sd/instances.json"
6
7  echo "Generating Prometheus instances.json from Terraform outputs..."
8
9  cd "$SHARED_DIR"
10
11  IPS=$(jq -r '.nat_vm_external_ips.value[]' terraform-outputs.json)
12
13  echo "[" > "$OUTPUT_FILE"
14
15  for ip in $IPS; do
16  | echo "{\"targets\": [\"${ip}:9100\"], \"labels\": {\"job\": \"node_exporter\"}},\" >> \"$OUTPUT_FILE"
17  done
18
19  # Remove last comma
20  sed -i '$ s/,,$//' "$OUTPUT_FILE"
21
22  echo "]" >> "$OUTPUT_FILE"
23
24  echo "instances.json generated successfully at $OUTPUT_FILE"
25

```

Рис. 13. Содержание Bash скрипта для подключения внешних узлов

3.4.4. Настройка отображения графиков

После подключения всех узлов к Prometheus следующим шагом стала настройка визуализации данных в Grafana. Этот инструмент позволяет создавать дашборды и наглядно отображать ключевые метрики, собираемые с виртуальных машин. Программа была установлена на той же машине, что и Prometheus, и работала с ним напрямую через встроенный datasource [6].

Визуализация настраивалась с помощью готовых шаблонов, которые уже содержали нужные графики и параметры. Один из них предназначен для node_exporter и автоматически распознаёт метрики нагрузки процессора, использования памяти, состояния файловых систем и сетевых интерфейсов. После подключения Prometheus как источника данных Grafana сразу начала отображать актуальную информацию со всех подключённых инстансов.

Использование механизма file_sd позволило автоматически добавлять в мониторинг все новые машины — их не приходилось прописывать вручную, Grafana сама начинала отображать метрики. Можно в реальном времени

следить за состоянием каждого узла и быстро реагировать на возможные перегрузки.

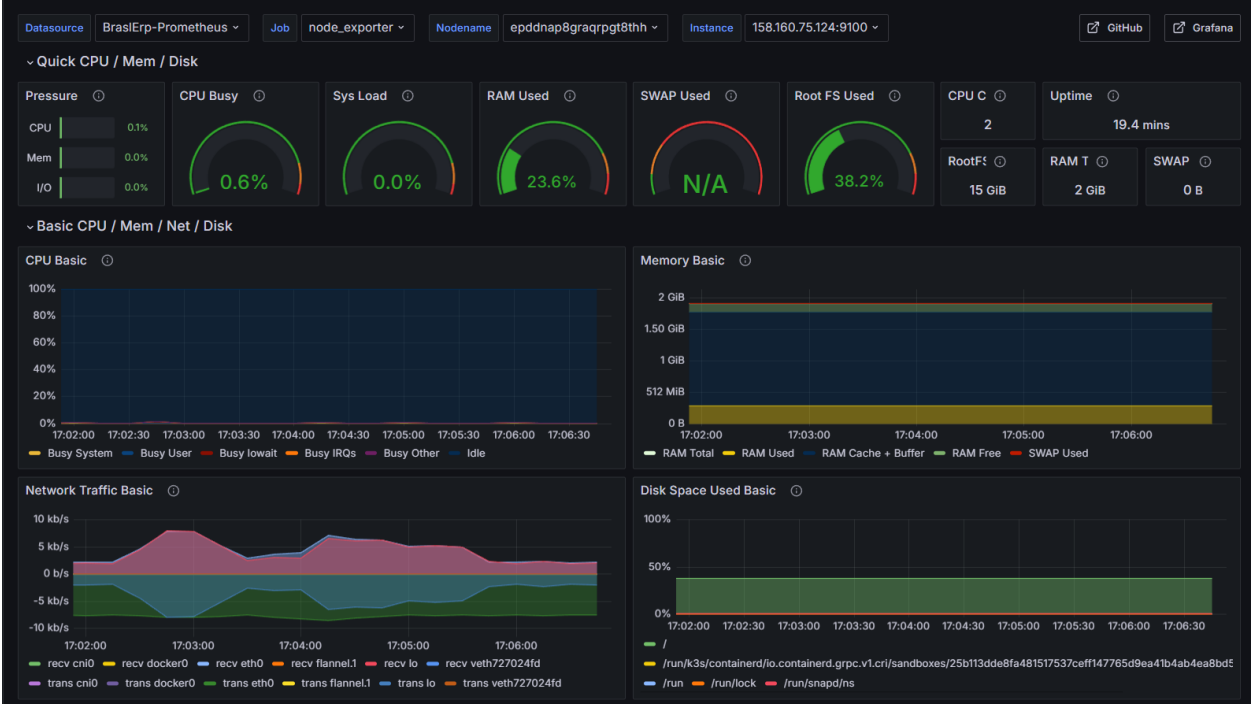


Рис. 14. Графики мониторинга мастер-сервера



Рис. 15. Графики мониторинга внешних узлов

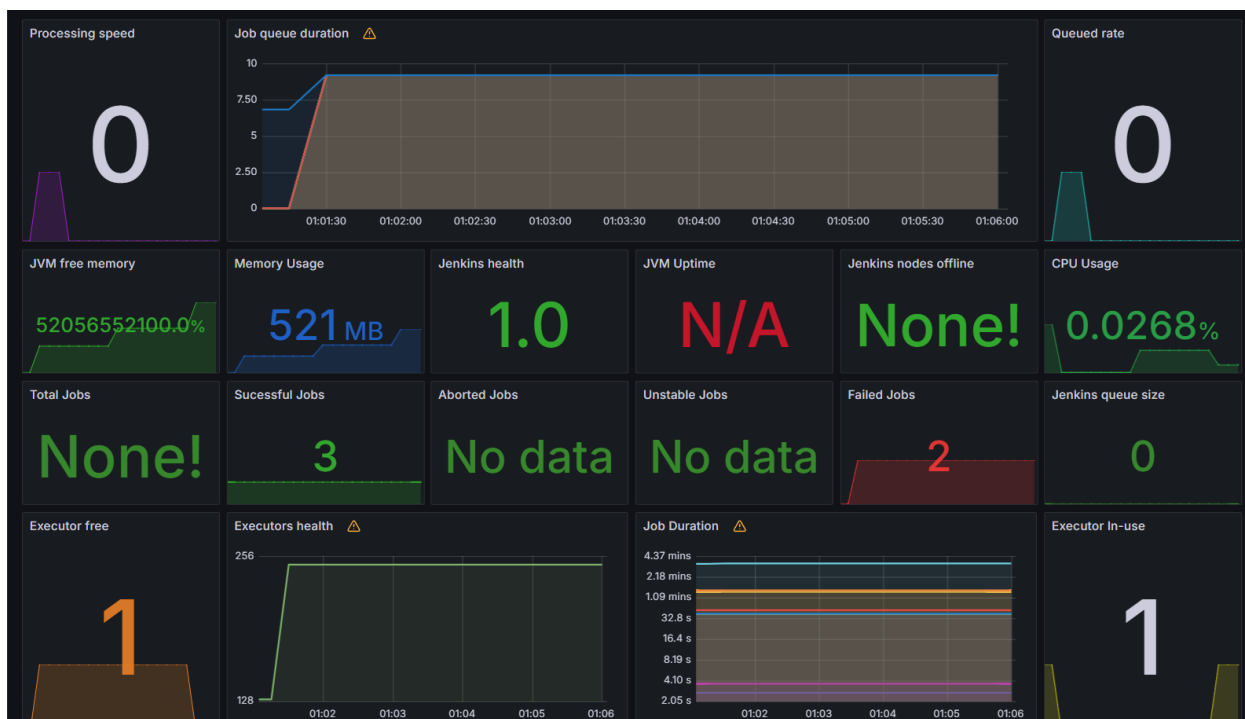


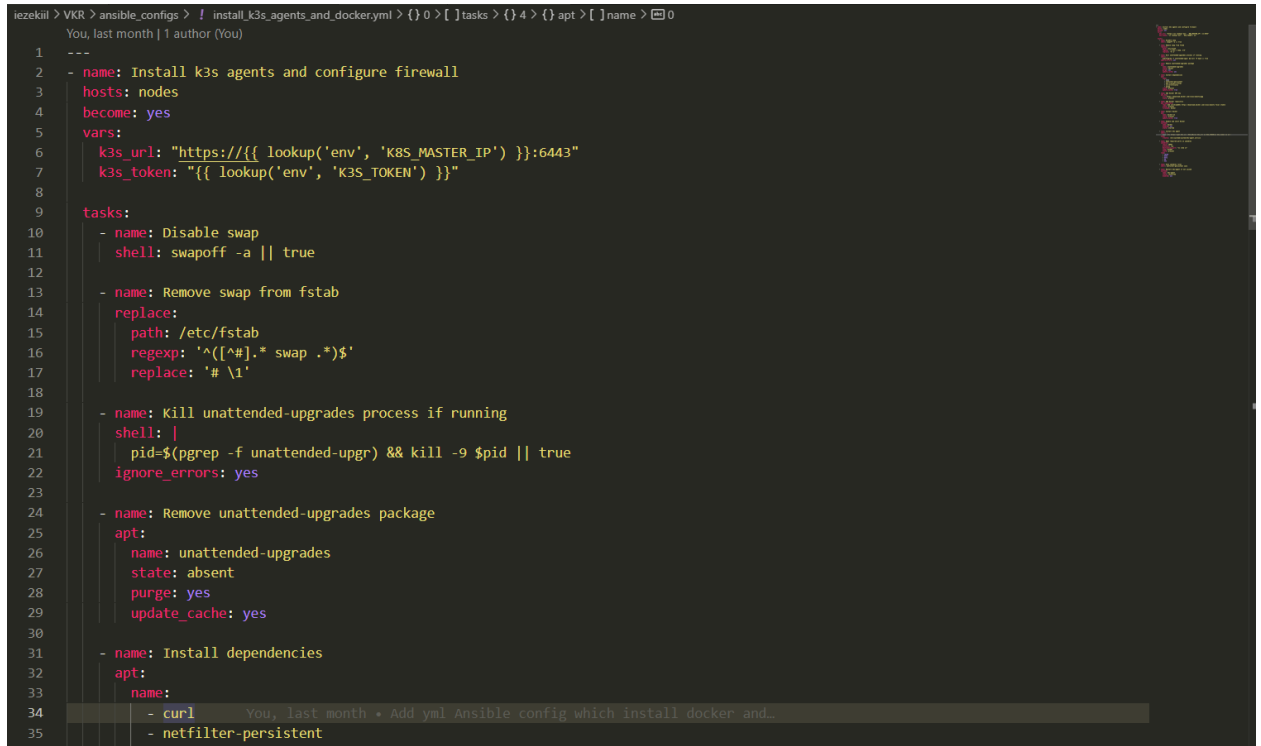
Рис. 16. Графики мониторинга программы Jenkins

3.5. Установка и настройка инструментов автоматизации

Следующим шагом стала настройка окружения на всех созданных машинах. Задача этого этапа — подготовить каждый узел так, чтобы он мог сразу принимать контейнеры, подключаться к кластеру и выполнять нужные действия без лишних операций вручную. Настройка начиналась с установки Docker — он необходим, чтобы запускать приложения в отдельных, изолированных средах. Это базовая часть, без которой невозможно продолжить работу с кластером. Установка проходила автоматически с помощью Ansible: система подключалась к каждой машине и выполняла нужные команды — устанавливала пакеты, добавляла пользователя в группу, запускала сервис и проверяла его работу [3].

После этого на каждой машине настраивался k3s — облегчённый вариант Kubernetes, хорошо подходящий для облачных решений, особенно когда узлы расположены в разных зонах. Установка делилась на две части: сначала один из узлов становился управляющим, после чего остальные подключались к нему как рабочие. Для каждого подключаемого узла

указывался внешний адрес — это позволяло обеспечить связь между машинами, даже если они находились в разных регионах. Все действия по установке были заранее описаны в отдельном сценарии Ansible, который автоматически применялся на всех узлах [3][7].



```
1 ---
2 - name: Install k3s agents and configure firewall
3   hosts: nodes
4   become: yes
5   vars:
6     k3s_url: "https://{{ lookup('env', 'K8S_MASTER_IP') }}:6443"
7     k3s_token: "{{ lookup('env', 'K3S_TOKEN') }}"
8
9   tasks:
10    - name: Disable swap
11      shell: swapoff -a || true
12
13    - name: Remove swap from fstab
14      replace:
15        path: /etc/fstab
16        regexp: '^(^#).* swap .*$'
17        replace: '# \1'
18
19    - name: Kill unattended-upgrades process if running
20      shell: |
21        pid=$(pgrep -f unattended-upgr) && kill -9 $pid || true
22      ignore_errors: yes
23
24    - name: Remove unattended-upgrades package
25      apt:
26        name: unattended-upgrades
27        state: absent
28        purge: yes
29        update_cache: yes
30
31    - name: Install dependencies
32      apt:
33        name:
34          - curl
35          - netfilter-persistent
```

Рис. 17. Содержание файла install_k3s_agents_and_docker.yml

Для гарантированной совместимости все машины изначально приводились к единому состоянию: проверялась версия ядра, загружались нужные модули, настраивались системные параметры. Это исключало ошибки в процессе и ускоряло настройку.

Вся установка и конфигурация выполнялась как отдельный шаг в Jenkins-конвейере, от момента создания машины до полной готовности к приёму рабочих нагрузок проходило всего несколько минут — без необходимости вручную заходить на каждый сервер.

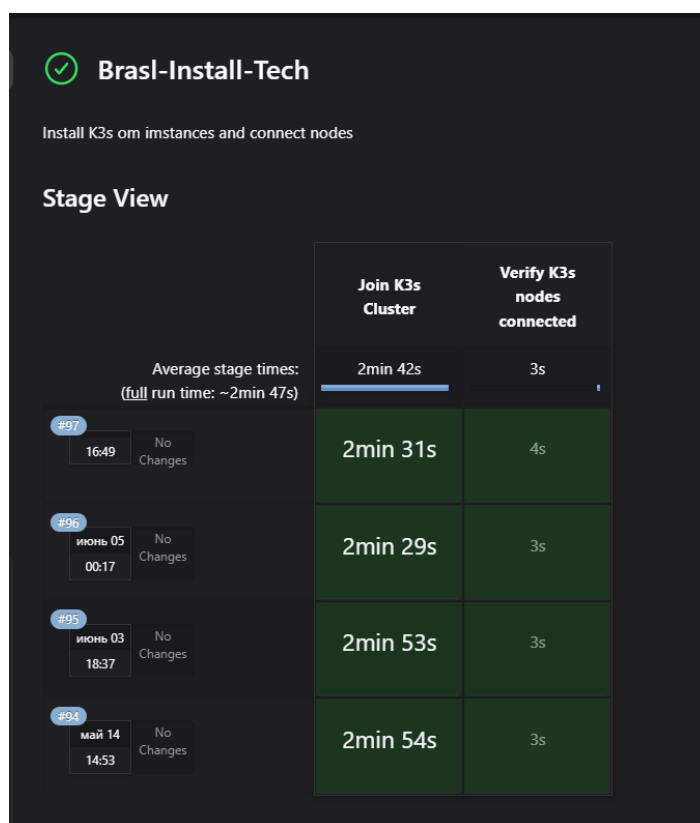


Рис. 18. Конвейер Jenkins по установке зависимостей

3.6. Разработка симуляции транспортного движения

Центральным элементом проекта стала симуляционная модель, имитирующая движение транспортных потоков на перекрёстке с четырьмя направлениями. Она была разработана на языке Python с использованием библиотеки Pygame, обеспечивающей визуализацию в реальном времени. Главной задачей системы стало моделирование поведения автомобилей и алгоритмов работы светофоров в условиях динамически меняющейся загрузки [8][9].

Разработка началась с создания геометрии перекрёстка и движения автомобилей. Все дороги, повороты и полосы были описаны как набор координат, по которым автомобили движутся пошагово. Для этого использовался модуль geometry.py, где задавались масштаб, координаты центра, полосы движения и пути. Автомобили создавались с разной

скоростью и типом движения, а их перемещение происходило с учётом столкновений, светофоров и других машин.

Светофорная логика была реализована в модуле `models.py`. Каждый светофор имел цикл включения и выключения, при этом его длительность зависела от текущей загруженности соответствующей полосы. Зоны детекции, определяющие количество машин перед перекрёстком, описывались вручную и использовались для оценки плотности трафика в реальном времени. Алгоритм переключения фаз адаптировался в зависимости от текущей ситуации, реализуя элементы интеллектуального регулирования [10].

Основной цикл симуляции находился в `simulation.py`. Он выполнял все ключевые действия: генерацию новых машин, обновление состояния светофоров, обработку движения и пересечений, а также определение приоритетов. Система работала в непрерывном режиме и позволяла наблюдать за изменением поведения в зависимости от времени суток и уровня загруженности — например, в моменты "пиковой нагрузки" поток автомобилей увеличивался, а длительность зелёного сигнала автоматически подстраивалась.

Для запуска симуляции использовался отдельный Python-файл, в котором происходила инициализация компонентов и запуск основного потока. Код был написан модульно и с возможностью масштабирования, что позволило позже адаптировать его под контейнеризацию и запуск в облачной инфраструктуре [8].

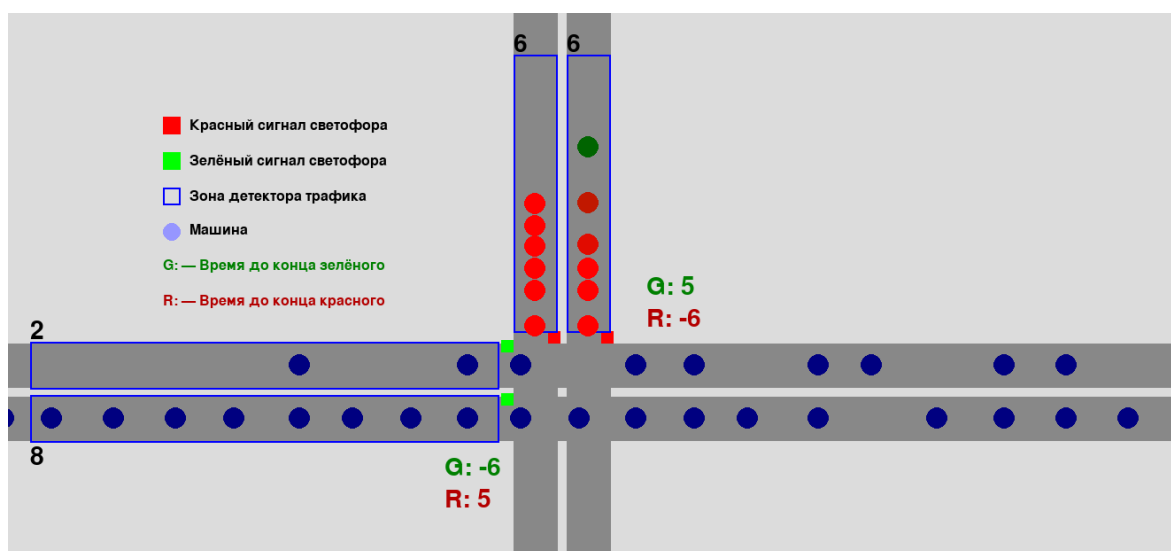


Рис. 19. Симуляция АСУ светофорами

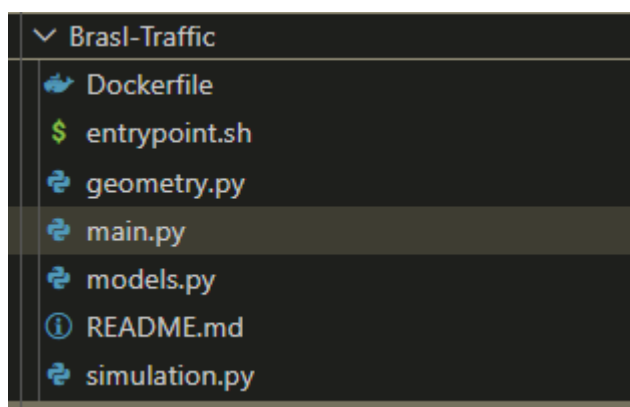


Рис. 20. Директория с файлами проекта симуляции

3.7. Сборка Docker-образа и публикация в Docker Hub

После завершения разработки симуляционной модели следующим шагом стала её контейнеризация. Это позволило запускать систему в изолированной среде и обеспечить переносимость между машинами. Для упаковки приложения использовался Docker, а образ впоследствии публиковался в Docker Hub, откуда его можно было развёртывать на любых узлах через Kubernetes.

Dockerfile для симуляции был создан вручную. В нём описывалась установка необходимых зависимостей Python, включая библиотеки Pygame, FastAPI, а также запуск noVNC для возможности удалённого подключения к

визуализации. Код симуляции копировался внутрь контейнера, где также настраивался рабочий каталог и команда запуска. Вся конфигурация была минимизирована с целью уменьшения итогового размера образа [4].

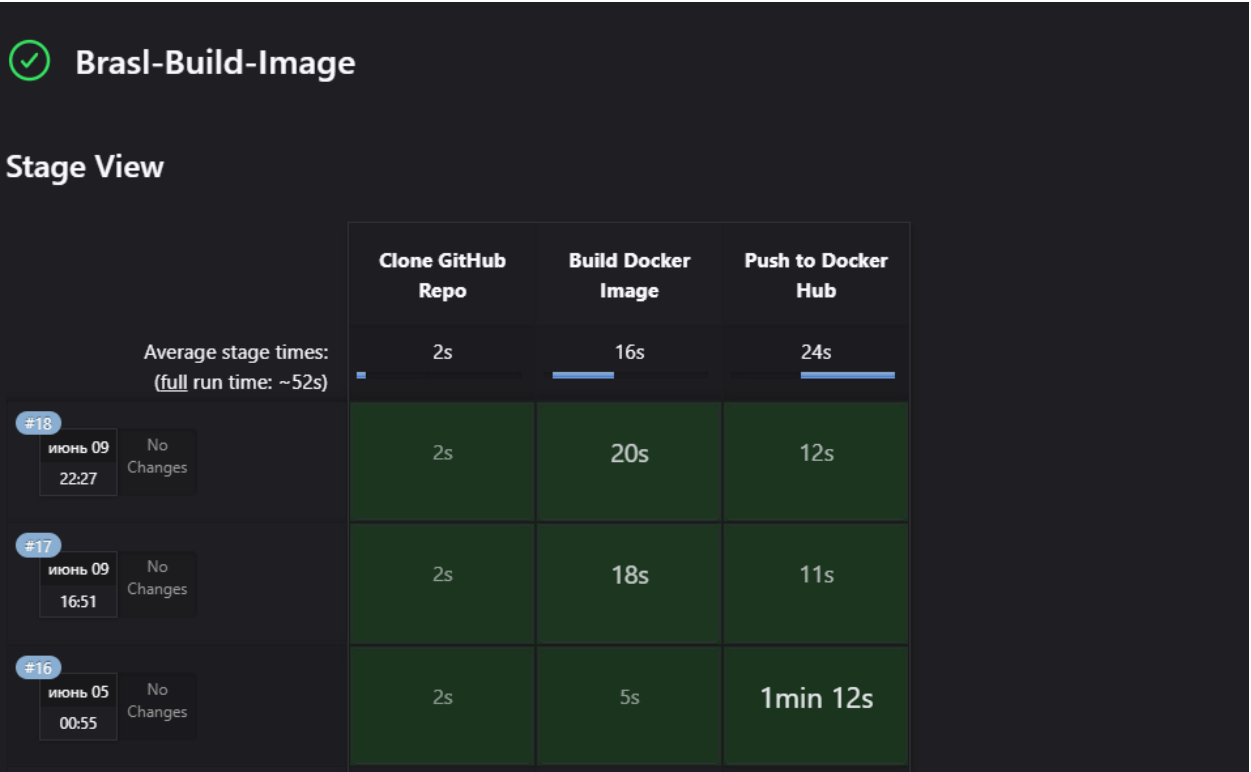


Рис. 21. Конвейер Jenkins для сборки образа приложения-симуляции

Сборка происходила через отдельный конвейер в Jenkins. На одном из этапов автоматически выполнялась команда `docker build`, затем — авторизация в Docker Hub, и завершалась публикацией образа через `docker push`. Благодаря этому новый образ можно было мгновенно использовать для развёртывания в k3s без дополнительных манипуляций.

Каждая сборка сопровождалась тэгом версии, что позволяло точно отслеживать изменения и возвращаться к предыдущим вариантам при необходимости. При успешной публикации Jenkins сохранял информацию о версии, чтобы на следующем этапе её можно было использовать в Kubernetes-манифестах.

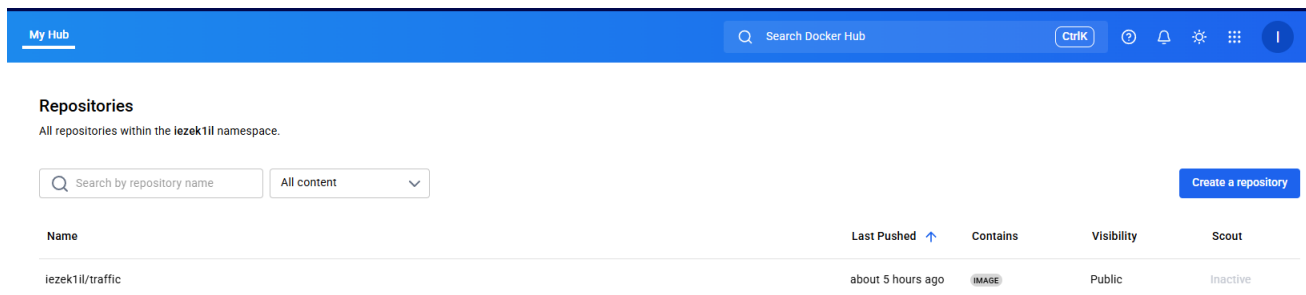


Рис. 22. Публичный образ приложения в Docker Hub

```
iezek11l > Brasl-Traffic > Dockerfile > ...
1 FROM python:3.11-slim
2
3 ENV DEBIAN_FRONTEND=noninteractive
4 ENV DISPLAY=:1
5 ENV SCREEN_SIZE=1800x1000x24
6
7 # Обновления и зависимости
8 RUN apt-get update && apt-get install -y --no-install-recommends \
9     xvfb x11vnc fluxbox curl git python3-pip procps \
10    libx11-6 libxext6 libxrender1 libxrandr2 libxv1 libxss1 libgl1-mesa-glx \
11    && pip install --no-cache-dir pygame \
12    && git clone --depth 1 https://github.com/novnc/novnc.git /opt/novnc \
13    && git clone --depth 1 https://github.com/novnc/websockify /opt/novnc/opts/websockify \
14    && apt-get purge -y git curl \
15    && apt-get clean \
16    && rm -rf /var/lib/apt/lists/* /root/.cache
17
18 # Копирование приложения
19 COPY . /app
20 WORKDIR /app
21
22 EXPOSE 6080
23
24 COPY entrypoint.sh /entrypoint.sh
25 RUN chmod +x /entrypoint.sh
26 CMD ["/entrypoint.sh"]
27
```

Рис. 23. Dockerfile для приложения

3.8. Развёртывание подов симуляции на удалённых k3s-нодах

Финальным этапом всей цепочки автоматизации стало разворачивание симуляции на распределённых узлах, объединённых в кластер через k3s. После того как Docker-образ был опубликован в Docker Hub, его стало возможно использовать для создания подов на любой из нод, вне зависимости от зоны размещения или конкретного IP-адреса [5].

Для развёртывания использовались YAML-манифест Kubernetes, описывающий ресурс типа Deployment и связанный с ним Service. Каждый под запускал контейнер на основе образа симуляции, ранее собранного и загруженного. Чтобы обеспечить доступ извне, для сервиса использовался тип NodePort, благодаря чему каждый экземпляр симуляции был доступен по публичному IP-адресу соответствующей ноды.

```
jenkins@brasl-vkr-master:~$ cd /var/lib/jenkins/manifests/
jenkins@brasl-vkr-master:~/manifests$ ls
traffic-novnc.yaml
jenkins@brasl-vkr-master:~/manifests$ cat traffic-novnc.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: brasl-traffic
spec:
  replicas: 1
  selector:
    matchLabels:
      app: brasl-traffic
  template:
    metadata:
      labels:
        app: brasl-traffic
    spec:
      containers:
        - name: brasl-traffic
          image: iezek1il/traffic:v1.0.0
          ports:
            - containerPort: 6080
          securityContext:
            privileged: true
---
apiVersion: v1
kind: Service
metadata:
  name: brasl-traffic
spec:
  type: NodePort
  selector:
    app: brasl-traffic
  ports:
    - protocol: TCP
      port: 80
      targetPort: 6080
      nodePort: 30000
```

Рис. 24. Содержание файла traffic-novnc.yaml

Далее происходил запуск команды `kubectl apply`, которая размещала под по заданным нодам. Такой подход обеспечил не только равномерное распределение нагрузки, но и отказоустойчивость: если один узел выходит из строя, другие продолжают работу [2][7].

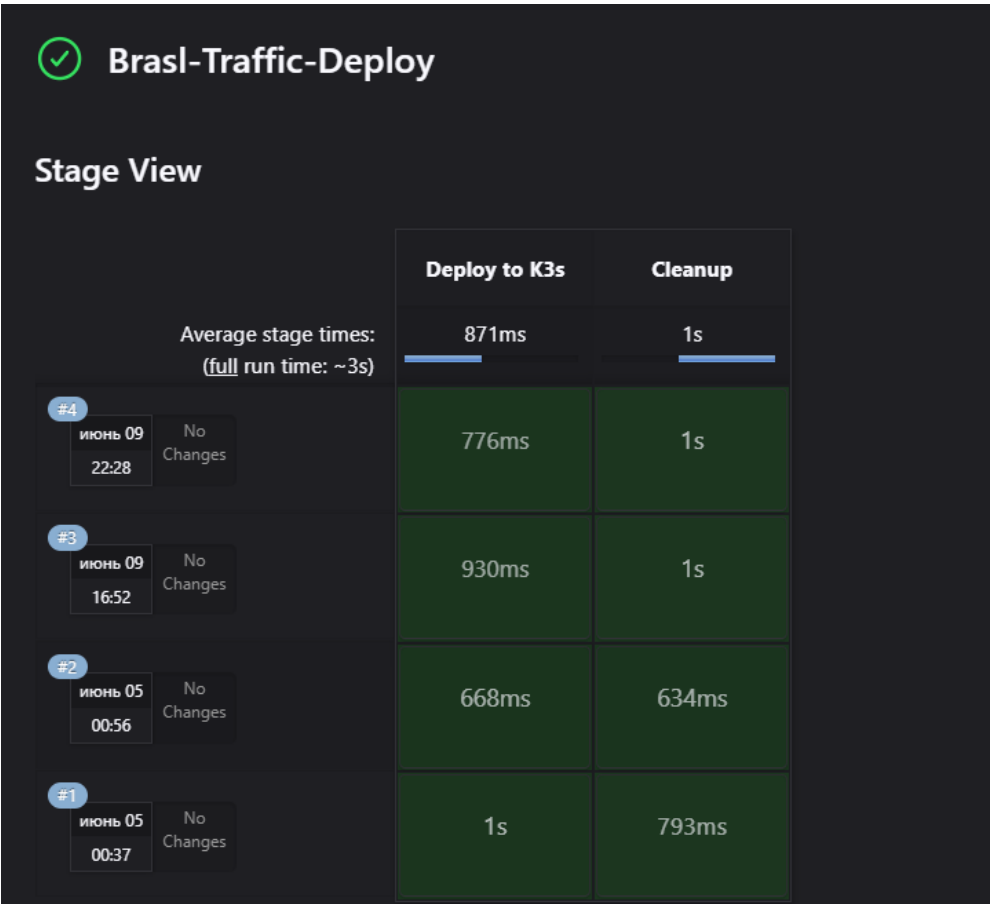


Рис. 25. Конвейер Jenkins по запуску под’а

Контейнеры с симуляцией запускались в привилегированном режиме, так как использовали Rugame и виртуальный X-сервер. Для доступа к визуализации применялся noVNC, что позволяло открывать интерфейс симуляции прямо в браузере с любого внешнего устройства. Всё это делало развёрнутые поды полностью самодостаточными и доступными для тестирования, демонстрации или анализа работы алгоритмов.

```
jenkins@brasl-vkr-master:~$ kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
brasl-vkr-master    Ready    control-plane,master   36d   v1.32.4+k3s1   158.160.145.91   158.160.145.91   Ubuntu 24.04.2 LTS   6.8.0-60-generic   containerd://2.0.4-k3s2
epdnugt3sf8jqls127d Ready    <none>    2m56s v1.32.5+k3s1   192.168.11.23    51.250.26.242    Ubuntu 22.04.4 LTS   5.15.0-101-generic   containerd://2.0.5-k3s1.32
fhmm1mo174nie5itd3kd Ready    <none>    2m56s v1.32.5+k3s1   192.168.10.22    51.250.71.38     Ubuntu 22.04.4 LTS   5.15.0-101-generic   containerd://2.0.5-k3s1.32
fv4rms93ef3oe7dsqd7b Ready    <none>    2m56s v1.32.5+k3s1   192.168.12.12    130.193.56.188    Ubuntu 22.04.4 LTS   5.15.0-101-generic   containerd://2.0.5-k3s1.32

jenkins@brasl-vkr-master:~$ kubectl get pods -o wide
NAME                READY    STATUS    RESTARTS   AGE   IP            NODE           NOMINATED NODE   READINESS GATES
brasl-traffic-9689856f9-8d5m2 1/1      Running   1 (29m ago)  5h19m  10.42.0.181   brasl-vkr-master   <none>           <none>
```

Рис. 26. Вывод работающих нод и под’а в k3s

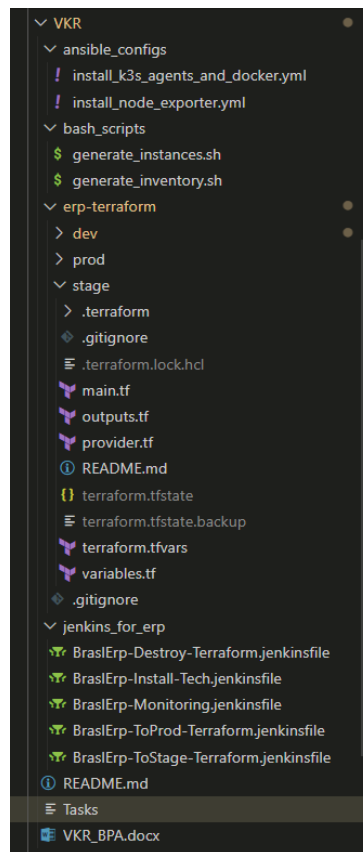


Рис. 27. Директория с файлами проекта

ЗАКЛЮЧЕНИЕ

В результате проделанной работы была разработана система автоматизированного развёртывания симуляции транспортного движения с элементами адаптивного управления. Проект охватывает весь цикл: от создания облачной инфраструктуры до запуска контейнеризованного приложения на удалённых узлах в составе k3s-кластера.

Особое внимание было уделено автоматизации процессов с применением инструментов Terraform, Jenkins, Ansible и Docker. Развёртывание инфраструктуры и установка необходимых компонентов происходят полностью без участия пользователя, что значительно снижает вероятность ошибок и повышает устойчивость системы. Мониторинг реализован через Prometheus и Grafana, с автоматическим подключением всех узлов [16].

Разработанная симуляция, написанная на Python с использованием Pygame, моделирует работу перекрёстка с учётом плотности потока и адаптации светофорных фаз. Приложение упаковано в Docker-образ и доступно для развёртывания через Kubernetes-манифесты. Визуализация осуществляется через noVNC в браузере.

Система может применяться как учебный стенд, как основа для демонстрации CI/CD-процессов, а также как платформа для дальнейших исследований в области управления транспортными потоками.

Репозитории с программным кодом системы и приложения-симуляции:

- <https://github.com/BruhSlave/VKR>
- <https://github.com/BruhSlave/Brasl-Traffic>